

Rapport de performance — MicroCRM

Fiche du document	Valeur
Projet	MicroCRM (Orion) — P5, Expert DevOps, « Gérez le cycle de vie de développement logiciel »
État décrit	version 1.0.1 , en production depuis le 5 octobre 2026
Périmètre	chaîne CI/CD, qualité et sécurité du code, déploiement Kubernetes, supervision (logs, traces, alertes), release
Destinataires	le CTO d'Orion, les équipes de développement et d'exploitation, le jury

Ce rapport synthétise ce que le projet a produit, ce qu'il a mesuré, et ce qu'il n'a pas atteint. Il est destiné à un lecteur qui n'ouvrira pas le dépôt : chaque affirmation renvoie au document technique qui la porte, et tous les chiffres cités ont été relevés sur cette installation, jamais estimés. Les numéros de pipeline cités se vérifient sur le projet GitLab public ; sauf mention contraire, les chiffres ont été relevés le 5 octobre 2026 par son API, sur le cluster et dans les artefacts des jobs.

Le document jumeau, `documentation-infrastructure.md`, décrit l'architecture et les procédures. Celui-ci décrit les résultats.

Synthèse pour décideur

Cette page se lit seule. Les sigles sont expliqués au §10.

Ce qui existe

Orion disposait d'une application, MicroCRM, sans chaîne de livraison. Elle en a maintenant une : à chaque modification du code, un **pipeline** (une suite de contrôles automatiques) teste l'application, cherche les failles de sécurité connues, fabrique les **images** (les paquets prêts à déployer), puis les installe sur un environnement de recette (« staging ») et un environnement de production. Une **supervision** rassemble les journaux et les temps de réponse de l'application dans des tableaux de bord et déclenche des **alertes**. La version **MicroCRM 1.0.1** a parcouru toute la chaîne jusqu'à la production le 5 octobre 2026.

Ce que ça change pour Orion

- **Une mise en production n'est pas un geste d'expert.** Elle se déclenche d'un clic, et le retour à la version précédente aussi. Les deux ont été joués en production le 23 septembre ; la version 1.0.1 a été publiée par ce chemin.
- **Ce qui part en production est exactement ce qui a été testé.** Une release ne reconstruit rien : les images 1.0.1 ont la même empreinte que celles que le pipeline a testées et scannées.

- **Une faille connue arrête la livraison avant la mise en ligne**, et non après. Les images en service sont passées par cette porte sans aucune faille haute ou critique.
- **On sait ce que la livraison vaut**. Quatre indicateurs reconnus (DORA) mesurent sa cadence et sa fiabilité, sans rien embellir — y compris quand l'échec vient de la machine et non du code.
- **Un incident se voit**. Huit règles d'alerte couvrent la disponibilité, la performance et la sécurité ; chacune a été déclenchée volontairement pour prouver qu'elle fonctionne.

Les cinq chiffres qui comptent

Chiffre	Ce qu'il veut dire
8 déploiements réussis sur 15 tentatives	Sur 30 jours. Neuf tentatives sur quinze (60 %) comptent comme des échecs. Trois d'entre elles sont tombées sur un cluster arrêté : c'est la machine qui a lâché, pas le code — mais elles comptent.
4,76 h entre une modification et sa mise en ligne	Valeur médiane sur 8 livraisons. Elle contient l'attente du clic humain, et pour l'une d'elles un week-end entier.
0 faille critique ou haute	Sur les images en service (version 1.0.1), relevé par le scan du pipeline avant leur publication.
8 alertes sur 8 déclenchées puis rétablies	Essai du 2 octobre. Les alertes s'affichent dans l'outil de supervision ; elles n'envoient ni courriel ni message.
16,1 secondes de coupure	Durée d'indisponibilité si le serveur applicatif tombe. Un déploiement, lui, ne coupe pas le service (99,75 % de requêtes servies pendant un déploiement suivi d'un retour arrière).

Les risques qui restent

1. **La plateforme est le maillon faible, pas le code**. Le runner et le cluster tournent sur le moteur Docker d'un poste, qui a décroché quatre fois pendant la phase de release (2 au 5 octobre). Chaque fois, la relance a suffi ; chaque fois, il a fallu quelqu'un pour la faire.
2. **La « production » est une démonstration**. Les deux environnements tournent sur un même ordinateur de développement, qui partage 8 Go de mémoire avec d'autres projets. Il n'existe pas de production réelle.
3. **Une panne du serveur applicatif coupe le service 16 secondes et efface les données**, parce que la base de données vit en mémoire.
4. **Personne n'est prévenu d'une alerte s'il ne regarde pas l'écran**, et les journaux de la production ne sont pas collectés (ses temps de réponse, eux, le sont).
5. **Douze failles du socle Spring sont acceptées sous condition, jusqu'au 31 décembre 2026**. Leur correctif gratuit n'existe que dans la version majeure suivante du socle.

Les décisions attendues

Décision	Pourquoi maintenant	Détail
Doter la chaîne d'un cluster et d'un runner qui ne dépendent pas d'un poste	Trois des neuf échecs comptés sur le mois viennent d'un cluster arrêté ; la chaîne n'est pas plus fiable que lui	§7.3
Planifier la migration vers Spring Boot 4 avant le 31 décembre 2026	À cette date, les exceptions de sécurité expirent et la livraison se bloque	§4.4
Remplacer la base en mémoire par PostgreSQL	Supprime la coupure de 16 secondes et la perte de données	§7.4
Choisir un canal d'alerte (messagerie, astreinte)	Une alerte que personne ne reçoit ne protège pas	§6.5
Financer un véritable environnement de production	Tout ce qui est mesuré ici l'est sur un poste de développement	§8

1. L'essentiel, y compris ce qui fâche

Le projet livre une chaîne complète. Deux images Docker construites en multi-étapes. Un pipeline GitLab CI de 10 étapes et 39 jobs, exécuté par un **runner auto-hébergé** (le quota de minutes du Free Tier ne suffit pas à cette chaîne). Des manifestes Kubernetes en overlays Kustomize, doublés d'un chart Helm. L'infrastructure décrite en Terraform et le poste provisionné par Ansible. Une stack ELK qui collecte réellement les logs de l'application et les traces de l'API, cinq tableaux de bord et huit règles d'alerte versionnés. Et deux collecteurs écrits pour ce projet : l'un pour les indicateurs DORA, l'autre pour les constats de sécurité.

Sur les trente derniers jours, huit déploiements ont abouti sur quinze tentatives. Staging et production sont posés par la CI avec des images du registry GitLab. Le taux d'échec des changements est de **60 %** : le chemin automatisé existe, il n'est pas encore fiable, et trois journées d'observations — les 22 et 23 septembre, le 5 octobre — ne font pas une cadence de livraison. Les échecs portent tous sur l'accès au cluster, jamais sur l'application : un cluster injoignable depuis un conteneur ou des droits insuffisants en septembre, un cluster arrêté le 5 octobre. **Une chaîne de livraison n'est pas plus fiable que la machine qui l'exécute.**

Les preuves de fonctionnement, jouées sur la chaîne réelle :

Date	Ce qui a été joué	Résultat	Où c'est établi
10 août 2026	Déploiement manuel de l'overlay <code>staging</code> sur minikube, incidents provoqués	rollouts en <code>0</code> , sondes observées sous kubelet, Ingress routé, rollback automatique sur échec réel	<code>K8S.md</code> §14

Date	Ce qui a été joué	Résultat	Où c'est établi
22 sept.	Premier <code>deploy-staging</code> depuis la CI	succès, images tirées du registry GitLab	<code>MONITORING.md</code> §9
22 sept.	Destruction puis reconstruction de l'environnement staging depuis le dépôt	Ansible <code>ok=23 changed=0</code> , Terraform, rollout en 11 s, API servie	<code>RELEASE.md</code> §9.5
23 sept.	<code>rollback-production</code> puis redéploiement	version précédente rétablie, puis redéployée	<code>MONITORING.md</code> §9
2 oct.	Déclenchement volontaire des 8 règles d'alerte	8 déclenchées, 8 rétablies seules	§6.5
5 oct.	Release 1.0.1 : pipelines <code>#2909284076</code> , <code>#2912362926</code> , <code>#2913490784</code>	images promues sans reconstruction (digests identiques), Release GitLab créée, production en 1.0.1	§7.3, <code>RELEASE.md</code> §7.5

La campagne manuelle du 10 août reste la seule à avoir exercé le **comportement en exploitation** — rollback sur image volontairement cassée, sondes sous kubelet, perte d'un pod — qu'aucun déploiement de CI ne réexerce.

Tableau de bord du projet

Domaine	Résultat mesuré	Où c'est établi
Pipeline	39 jobs, 10 étapes, toutes images d'outillage figées	<code>ARCHITECTURE.md</code> §4
Tests back	115 tests · 97,40 % lignes · 100 % branches	<code>back/</code> , job <code>test-back</code>
Tests front	112 tests · 100 % lignes · 90,24 % branches	<code>front/</code> , job <code>test-front</code>
Tests des scripts	430 assertions, sans cluster ni registry	<code>SCRIPTS.md</code>
Validation des manifestes	151 assertions, dont l'équivalence Kustomize ↔ Helm	<code>SCRIPTS.md</code> , <code>HELM.md</code> §6
Playbook Ansible	<code>ok=23 changed=0</code> sur deux exécutions consécutives	<code>ANSIBLE.md</code> §7
Terraform	3 environnements, <code>validate</code> et <code>plan</code> en <code>0</code>	<code>TERRAFORM.md</code>
Latence du front	p50 0,14 ms · p95 1,60 ms · p99 3,11 ms (105 req.)	<code>MONITORING.md</code> §5

Domaine	Résultat mesuré	Où c'est établi
Latence de l'API	p95 de 3,41 à 5,65 ms selon la route — conteneur local (§5.3)	MONITORING.md §10.4
Traces de l'API en service	staging et production	MONITORING.md §10
Logs collectés	92 175 documents au 2 octobre, 100 % du namespace de staging	k8s/elk/alerting/README.md
Alertes	8 règles , 3 familles, 8 déclenchées puis rétablies	k8s/elk/alerting/README.md
Constats de sécurité	0 CRITICAL , 0 HIGH sur les images en service (1.0.1)	artefacts de package-*
Disponibilité en déploiement	99,75 % sur 1 615 requêtes, coupure ≤ 0,20 s	K8S.md §14
Déploiements réussis en CI	8 sur 15 tentatives (30 j)	MONITORING.md §9
Release publiée	1.0.1 , promue sans reconstruction, en production	RELEASE.md §7.5

2. Les indicateurs DORA

2.1 Les quatre indicateurs, et ce qu'ils signifient

Les indicateurs DORA (du nom du programme de recherche *DevOps Research and Assessment*) sont quatre mesures qui décrivent la livraison logicielle par ses résultats. Deux mesurent la **vitesse**, deux la **stabilité** ; les lire ensemble empêche d'améliorer l'une en dégradant l'autre.

Indicateur	Ce qu'il mesure	Comment il est calculé ici	Ce qu'il dit à Orion
Fréquence de déploiement	Combien de fois par jour une version arrive en service	Jobs de déploiement réussis, divisés par les 30 jours de la fenêtre	La cadence réelle de livraison
Délai de mise en production	Le temps entre une modification et sa mise en service	Médiane, du commit de tête à la fin du job de déploiement réussi	La réactivité : combien de temps attend un correctif
Temps de rétablissement (MTTR)	Le temps pour retrouver un service sain après un échec	Médiane, d'un déploiement en échec au déploiement réussi suivant	La capacité à se relever

Indicateur	Ce qu'il mesure	Comment il est calculé ici	Ce qu'il dit à Orion
Taux d'échec des changements	La part des déploiements qui échouent ou qu'il faut annuler	Échecs directs et réussites suivies d'un rollback, divisés par les tentatives	La fiabilité de ce qu'on livre

Pourquoi ceux-là. Ils se calculent depuis l'historique des pipelines, que le projet produit de toute façon : aucune saisie manuelle, donc aucun chiffre arrangé. Et ils mesurent ce qui intéresse Orion — des versions qui arrivent en service sans casse — plutôt que l'activité de l'équipe.

2.2 Comment ils sont produits

Les quatre indicateurs sont calculés par `scripts/ci/collect_dora.py`, en Python standard et sans aucune dépendance. Les métriques DORA natives de GitLab sont réservées aux offres payantes ; sur le Free Tier, il faut les calculer soi-même depuis l'API. Le projet est mesurable **sans jeton**, parce que le dépôt GitHub se miroite vers un projet GitLab public où le pipeline tourne réellement. Le job `dora-metrics` exécute le collecteur sur `develop`, `main` et les tags, et publie `reports/dora.json` en artefact.

Le collecteur est testé sur des **fixtures**, c'est-à-dire des réponses d'API enregistrées, et non contre le réseau : un test qui dépend d'un service tiers échoue les jours où ce service est lent, et on finit par ne plus le croire. Deux jeux coexistent. Le premier est réel : 44 pipelines enregistrés tels quels, sans aucun déploiement réussi. Le second, explicitement nommé `dora-scenario-fabrique`, contient ce que le jeu réel n'offre pas : des déploiements réussis. Sans lui, les formules du délai et du temps de rétablissement ne seraient empruntées par aucun test.

2.3 Les valeurs

Indicateur	Référence (23 septembre 2026, 50 pipelines)	Actuel (5 octobre 2026, 67 pipelines)	Observations
Fréquence de déploiement	0,1667 par jour	0,2667 par jour	5 → 8
Délai de mise en production (médiane)	1,38 h (min 0,64 / max 4,87)	4,76 h (min 0,64 / max 40,88)	5 → 8
Temps de rétablissement (MTTR, médiane)	2,14 h (min 0,09 / max 4,18)	2,21 h (min 0,09 / max 4,18)	2 → 4
Taux d'échec des changements	66,67 % (6 sur 9)	60 % (9 sur 15)	9 → 15

La référence est le premier relevé où les quatre indicateurs ont une valeur. La valeur actuelle a été calculée après le dernier déploiement de la release 1.0.1, contre l'API réelle, puis injectée dans l'index `microcrm-dora`. Sur la fenêtre : quinze jobs de déploiement exécutés, huit réussis, sept en échec,

et les deux rollbacks du 23 septembre.



Ce qu'il faut y lire : les quatre tuiles du haut sont les quatre indicateurs ; la chronologie montre que toute l'activité tient sur trois journées, et que les trois échecs du 5 octobre (en rouge, à droite) sont aussi nombreux que les réussites du même jour. Le texte d'aide en tête du tableau, versionné dans `dora.ndjson`, parle de « deux journées » : il est à mettre à jour.

Analyse. Une lecture honnête de ces quatre valeurs tient en cinq remarques.

Le taux d'échec compte des pannes de la plateforme. Trois `deploy-production` en échec sont tombés sur un cluster arrêté : aucun n'a rien écrit dans le namespace de production, et la même image est passée ensuite sans modification. Ce ne sont donc pas des changements défectueux.

DORA les compte quand même, et le collecteur ne peut pas les distinguer : GitLab les classe tous en `script_failure`, comme un échec de l'application. Sans eux, le taux serait de 50 % (6 sur 12). Le chiffre retenu reste 60 % : retirer après coup les mesures qui gênent serait exactement ce que ces indicateurs servent à empêcher. La lecture juste est que **le poste de développement, utilisé comme environnement, est la première cause d'échec de livraison mesurée.**

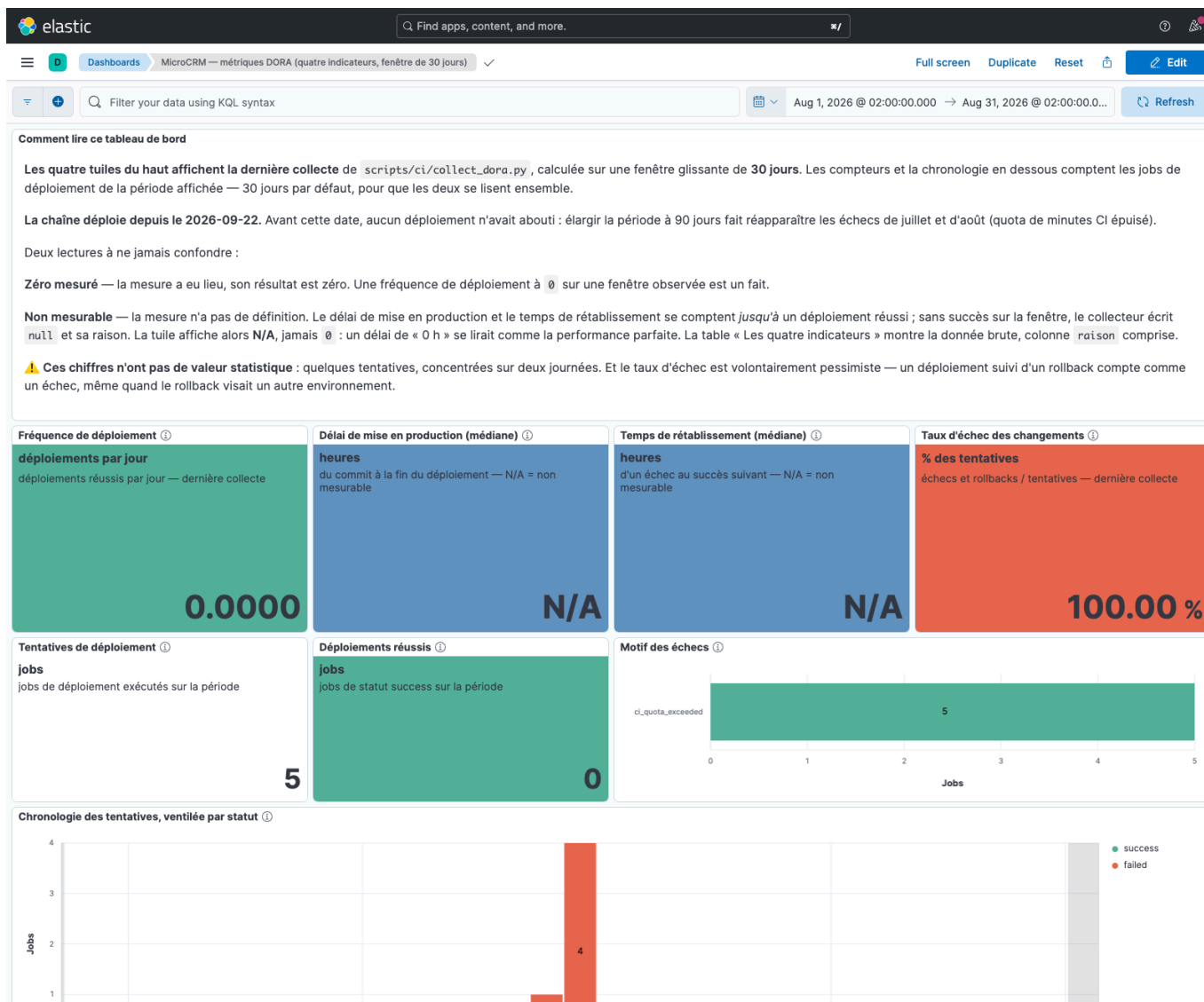
Le recul est de trois jours, pas de trente. Les huit déploiements réussis tiennent sur les 22 et 23 septembre et le 5 octobre ; la fréquence de 0,2667 par jour est une moyenne sur une fenêtre dont l'essentiel est vide. Un taux d'échec sur 15 tentatives et un MTTR sur 4 observations décrivent des faits, pas des tendances.

Le délai de mise en production mesure surtout un geste humain, et un week-end. Les deux jobs de déploiement sont `when: manual`. Le maximum de 40,88 h est un commit fusionné un samedi et déployé le lundi. La valeur mesure l'attente du clic, pas la durée du pipeline (environ 20 minutes de jobs automatiques sur `develop`).

Le temps de rétablissement mesure ici la relance d'un cluster. Ses deux dernières observations (4,17 h et 0,25 h) couvrent le temps de relancer minikube, pas de réparer l'application.

La fenêtre glisse. Sans nouveau déploiement, les réussites de septembre sortent de la fenêtre de 30 jours à partir du 22 octobre, celles du 5 octobre le 4 novembre ; la fréquence retombe alors à 0 et le délai redevient non mesurable. C'est l'effet attendu d'un indicateur glissant.

Zéro mesuré et absence de donnée ne sont pas la même chose. C'est la règle qui gouverne tout le collecteur. Sur une fenêtre sans déploiement réussi, `deployment_frequency` vaut 0.0 — une mesure — tandis que `lead_time_for_changes` vaut `null`, accompagné de sa raison : il n'existe aucune arrivée en production vers laquelle mesurer un délai. Le rendre en 0 afficherait un délai nul, c'est-à-dire **la performance parfaite**, là où il n'y a pas eu de mise en production. Un test de `run_tests.sh` échoue si un indicateur sans donnée ressort en 0, et le tableau de bord respecte la même règle :



Ce qu'il faut y lire : sur une période sans aucun succès, les deux indicateurs qui n'ont pas de définition affichent « N/A », jamais « 0 ».

⚠ Quatre limites à connaître sur cette collecte.

- **Le job s'exécute à un moment fixe.** `dora-metrics` tourne avant les déploiements manuels de son propre pipeline : il ne voit pas le déploiement qui le suit. La valeur de référence est donc celle d'un recalcul fait après le dernier déploiement.
- **Le job n'alimente pas le tableau de bord.** Il n'envoie rien à Elasticsearch, qu'un runner ne peut pas joindre ici. L'index `microcrm-dora` est alimenté à la main depuis le poste.
- **Le collecteur ne distingue pas une panne de la plateforme d'un changement défectueux.** Il lit le statut et le motif d'échec que GitLab donne au job ; un cluster injoignable et un manifeste faux y portent le même `script_failure`.
- **Le comptage des rollbacks est pessimiste, et assumé tel quel.** Un déploiement réussi puis annulé compte comme un échec, conformément à la définition DORA, **mais le rattachement d'un rollback à un déploiement est purement chronologique** : un rollback qui a échoué compte quand même comme une annulation. Le détail, et ce qu'il faudrait pour le corriger, sont dans `MONITORING.md` §9.3.

La liste des pipelines après la release :

The screenshot shows the GitLab Pipelines interface for the project 'G-rez-le-cycle-de-vie-de-d-veloppement-logiciel'. The left sidebar contains navigation links for 'Projets', 'Épinglé', 'Membres', 'Éléments de travail', 'Branches', 'Requêtes de fusion', 'Pipelines', 'Feature catalog', 'Gestion', 'Planification', 'Code', 'Compilation', 'Jobs', 'Planifications de pipeline', 'Artéfacts', 'Déploiement', 'Opération', 'Surveillance', and 'Analyse'. The main content area displays a table of pipeline runs.

État	Pipeline	Créé par	Étapes	Actions
Bloqué	#2913490784 Merge pull request #36 from TedPasquie... v1.0.1 → 88a216b8		✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓	▶ ⬇
Bloqué	#2912362926 Merge pull request #36 from TedPasquie... main → 88a216b8		✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓	▶ ⬇
Réussi	#2909284076 Merge pull request #35 from TedPasquie... develop → 5296658a		✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓	▶ ⬇
Annulé	#2909283107 Merge pull request #34 from TedPasquie... develop → 281e52fb		⏸ ⏸ ⏸ ⏸ ⏸ ⏸ ⏸ ⏸	▶ ⬇
Annulé	#2909282298 Merge pull request #33 from TedPasquie... develop → 911a7ad6		⏸ ⏸ ⏸ ⏸ ⏸ ⏸ ⏸ ⏸	▶ ⬇
Annulé	#2909281689 Merge pull request #32 from TedPasquie... develop → 3fa7d455		⏸ ⏸ ⏸ ⏸ ⏸ ⏸ ⏸ ⏸	▶ ⬇
Annulé	#2909280799 Merge pull request #31 from TedPasquie... develop → fe16acf2		⏸ ⏸ ⏸ ⏸ ⏸ ⏸ ⏸ ⏸	▶ ⬇
Réussi	#2909274796 chore(release): prépare la version 1.0.1 feature/pipeline-release-et-rapports → 51d4627b		✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓	⬇
En échec	#2909274793 feat(monitring): ajoute les règles d'alert... feature/alerting-kibana → c97e13d3		✓ ✓ ✗ ✓ ✓ ✓ ✓ ✓	⬇
Réussi	#2909274786 docs: régénère les PDF livrables docs/mise-a-jour-des-livrables → f6be1e19		✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓	⬇
En échec	#2909274777 fix(security): fait réellement analyser les ... fix/jackson-databind-cve → fe7f9731		✗ ✗ ✗ ✗ ✗ ✗ ✗ ✓	⬇

⚠ « Bloqué » n'est pas un échec, et c'est la première chose à lire sur cette capture. C'est le statut d'un pipeline dont tous les jobs automatiques ont réussi et qui attend une action manuelle — ici les `terraform-apply-*` et `rollback-production`, qui sont des portes volontaires (§2.4) restées sans usage. Les pipelines « Annulé » n'ont exécuté aucun job : le pipeline du dernier commit, #2909284076, contient tout leur contenu. Les deux « En échec » sont des branches de travail avant fusion ; le même code, fusionné, passe dans les trois pipelines du haut.

2.4 « Ça marche sur un cluster local » n'est pas « ça arrive en production »

C'est la différence que ces quatre chiffres mesurent, et il vaut mieux l'énoncer que la laisser deviner.

Ce qui est **prouvé sur un cluster** : les manifestes s'appliquent, les pods démarrent, les sondes se comportent comme prévu sous un vrai kubelet, l'Ingress route par hôte, le CORS discrimine, un déploiement vers une image inexistante déclenche bien le rollback automatique, et un rollback ramène la version précédente réellement déployée.

Ce qui est **prouvé par la CI** : un runner, un registry privé authentifié, des variables protégées, l'accès au cluster par le tunnel de l'agent GitLab, et un enchaînement que personne ne pilote entre le clic et le rollout. Les pods de staging et de production tirent leurs images du registry GitLab par `imagePullSecrets`.

Ce qui **n'a pas lieu** : qu'un commit poussé sur `develop` ou `main` se retrouve en marche sans intervention humaine. Les deux jobs de déploiement sont `when: manual` ; c'est un choix pour la production, et l'action A3.1 du plan d'optimisation propose de le lever sur staging seulement.

Ce que la campagne locale porte seule : le comportement sous incident. Les déploiements de CI en échec l'ont été sur l'accès au cluster, et non sur l'application ; aucun n'a exercé une image cassée, la perte d'un pod ou une sonde en échec. **Les indicateurs DORA restent l'instrument qui refuse de confondre « le mécanisme fonctionne » et « les changements arrivent en production sans casse »** : à 60 % d'échec, ils ne flattent personne, et c'est pourquoi ils sont présentés tels quels.

3. Tests et couverture

3.1 Les quatre suites

Suite	Volume	Couverture	Job CI
Back (JUnit, JaCoCo)	115 tests	97,40 % lignes · 100 % branches	<code>test-back</code>
Front (Karma, LCOV)	112 tests	100 % lignes · 90,24 % branches	<code>test-front</code>
Scripts d'automatisation	430 assertions	—	<code>test-scripts</code>
Manifestes et chart	151 assertions	—	<code>lint-helm</code>

Les quatre jobs sont verts dans les trois pipelines de la release (`#2909284076`, `#2912362926`, `#2913490784`), où GitLab affiche les 115 tests du back. `run_tests.sh` rend « 430 test(s) OK, 0 en échec » et `validate_k8s.sh` « 151 test(s) OK » (174 avec `--autotest`). Les 112 tests du front et leur couverture viennent d'une exécution locale de `ng test` le 2 octobre.

100 % des embranchements du back sont empruntés. Une couverture de lignes élevée ne dit rien des chemins d'erreur, et c'est là que se loge le risque. Le seuil dur du pipeline (`COVERAGE_MIN`) est fixé à **90 %** de lignes, contrôlé par `scripts/ci/check_coverage.py` dans le job `coverage-gate`, **bloquant** — un contrôle rapide, indépendant de la disponibilité de Sonar. Les tests de mutation (`MUTATION_MIN`, 80 %) posent la question que la couverture ne sait pas poser : ces tests vérifient-ils quelque chose ?

3.2 Ce que les tests d'infrastructure vérifient réellement

Deux suites ne testent pas du code applicatif, et ce sont celles qui ont le plus de valeur ici.

Les 430 assertions de `run_tests.sh` exercent les scripts de déploiement sans cluster ni registry : les vraies commandes `kubectl`, `docker`, `trivy` et `k6` sont remplacées par de faux programmes qui notent ce qu'on leur demande et renvoient le code de sortie voulu. On vérifie ainsi qu'un `rollout undo` est réellement déclenché quand un déploiement échoue, qu'il ne l'est **pas** quand tout va bien, que le scan d'une image précède son push, qu'aucune image n'est poussée si Trivy y trouve une CVE haute ou critique, et qu'aucun secret n'apparaît dans les logs. Elles couvrent aussi le collecteur de sécurité (56 assertions) et l'installation des alertes (33).

Les 151 assertions de `validate_k8s.sh` portent sur ce que Kustomize et Helm produisent réellement. La plus importante est la dernière : **le rendu du chart Helm est comparé objet par objet à celui de l'overlay Kustomize correspondant**, et l'écart n'est toléré que sur le seul label `app.kubernetes.io/managed-by`. Deux descriptions de la même application, c'est deux occasions de diverger ; c'est cette assertion qui rend la divergence visible en revue plutôt qu'au déploiement.

`validate_k8s.sh` est **auto-testé** : l'option `--autotest` rejoue ses assertions sur des rendus volontairement abîmés — Deployment renommé, ConfigMap fantôme, sonde sur un port inconnu, `runAsNonRoot` retiré, image en `latest` — et vérifie qu'elles échouent bien. Une assertion qui ne se déclenche jamais ne prouve rien.

3.3 Ce que les tests établissent

Trois résultats justifient le coût de ces suites.

- **Le scénario k6 se comporte comme le vrai client.** Spring Data REST ne renvoie la ressource créée par `POST /persons` que si le client envoie un en-tête `Accept` ; le front l'envoie, le scénario aussi. Un test qui ne vérifierait que le code HTTP laisserait passer un corps vide.
- **Le rollback de production vise une image réelle.** Le déploiement pose l'image réelle par un overlay Kustomize éphémère **avant** l'`apply` : chaque déploiement crée une seule révision, et la « révision précédente » est toujours une version réellement déployée, jamais le *placeholder* des manifestes. Vérifié sur cluster : deux déploiements, deux révisions, zéro placeholder, rollback sans argument en `0`.
- **La suppression d'un pod back coupe l'API 16,1 secondes.** Voir §5.4.

4. Qualité et sécurité du code

4.1 Les contrôles, et lesquels arrêtent le pipeline

Six outils sont branchés sur le cycle de vie, chacun voyant ce que les autres ne voient pas : un test unitaire ne détecte pas une CVE, un scan de CVE ne détecte pas une mauvaise pratique de code, et aucun des deux ne dit si l'application tient la charge.

Outil	Question à laquelle il répond	Étape	Bloquant ?
Checkstyle, ESLint, ShellCheck	La forme est-elle tenue ?	lint	oui
SonarQube	Bonnes pratiques, dette, couverture agrégée	quality	Quality Gate sur main seulement
SpotBugs	Bugs latents dans le bytecode	quality	non, rapport publié
OWASP Dependency-Check	CVE des dépendances Java (base NVD)	security	oui, score CVSS ≥ 7
Trivy	CVE du dépôt et des images, secrets, misconfigurations	security et package	oui, HIGH et CRITICAL
k6	L'API répond-elle, et assez vite ?	perf	oui pour le smoke, non pour la charge

Les contrôles de sécurité, de forme, de couverture et de mutation sont bloquants ; trois ne le sont pas, par choix. Les deux analyses SonarQube tournent à chaque pipeline mais n'attendent pas leur verdict : c'est le job `quality-gate`, qui interroge la *Quality Gate*, qui bloque, et il ne tourne que sur `main`. SpotBugs publie son rapport sans faire échouer le build (`ignoreFailures = true` dans `back/build.gradle`) : ses constats sont relus, pas opposés. Spotless et Prettier tournent dans les hooks locaux, pas en CI.

Le troisième contrôle non bloquant est **k6-load**. Ses mesures varient d'une exécution à l'autre sur un runner mutualisé, et un seuil dur y produirait des échecs sans rapport avec le code. Sur un runner dédié, il suffit de basculer son `allow_failure`. `k6-smoke`, lui, est bloquant, parce qu'il ne mesure pas une tendance mais un fait binaire : l'image qu'on s'apprête à déployer répond, ou elle ne répond pas. Deux autres jobs portent `allow_failure: true`, `notify-echec` et `dora-metrics` : ce ne sont pas des contrôles, et une notification ou une mesure en panne ne doit pas faire échouer une livraison.

Angle mort connu : aucun job ne lance `npm audit`. Les dépendances du front ne sont couvertes que par `trivy-fs`, qui lit le `package-lock.json`.

4.2 Trois choix qui rendent les contrôles de sécurité effectifs

Un contrôle bloquant peut être affaibli sans que rien ne le signale. Trois réglages l'empêchent, et chacun est vérifié en CI.

Dependency-Check analyse réellement les 82 dépendances du `runtimeClasspath`. Le plugin écarte par défaut les configurations « de test », qu'il reconnaît à leur nom, et Spring Boot fait hériter `runtimeClasspath` d'une configuration nommée `testAndDevelopmentOnly`. Sans `skipTestGroups = false` (`back/build.gradle`), le rapport sort vide et vert. Une porte bloquante qui ne lit rien ne se distingue pas, à l'œil, d'une porte qui n'a rien trouvé : c'est pourquoi le rapport en artefact est relu, et pourquoi un contrôle du nombre de dépendances analysées est recommandé (§7.2).

L'image est scannée avant d'être poussée. Le scan Trivy se fait **dans** `build_and_push.sh`, entre la construction et le push (option `--scan`, activée dans `package-back` et `package-front`), et un test de `run_tests.sh` vérifie cet ordre. Une image refusée n'atteint donc jamais le registry, où la promotion d'une release, qui ne vérifie que l'existence du tag, pourrait la publier.

Chaque scan laisse une trace exploitable. `scripts/ci/trivy_scan.sh` fait deux passages : un relevé au format JSON, puis la porte bloquante sous forme de tableau lisible. Les jobs `trivy-fs`, `package-back` et `package-front` publient ces rapports en artefacts (`reports/`, conservés une semaine, y compris quand le job échoue). Son code de sortie distingue un constat (2) d'un scan impossible (1).

4.3 L'état des vulnérabilités

Les images en service ne portent aucun constat HIGH ou CRITICAL. Les rapports Trivy des jobs `package-back` et `package-front`, dans les artefacts des pipelines `#2909284076` et `#2912362926`, portent **0 constat HIGH ou CRITICAL** sur `back:5296658a`, `front:5296658a`, `back:08a216b0` et `front:08a216b0` — système de base compris, et pour le back `microcrm.jar` et l'agent OpenTelemetry. Les deux dernières sont, à l'empreinte près, les images **1.0.1** en production. Trois versions sont forcées dans `back/build.gradle` au-dessus de celles que gère Spring Boot pour y parvenir : Tomcat 10.1.59, Jackson 2.21.7, Log4j 2.25.5.

Le collecteur `scripts/ci/collect_security.py` lit les rapports JSON de Trivy et de Dependency-Check, ainsi que les exceptions de `.trivyignore.yaml`, et les envoie dans l'index Elasticsearch `microcrm-security`. Le tableau de bord « sécurité » (§6.4) en est la lecture. L'index a été alimenté le 2 octobre ; son dernier scan de chaque source, toutes sévérités :

Source	Ouverts	CRITICAL	HIGH	MEDIUM	LOW	Exceptés
Dépôt (code, manifestes, configuration)	58	0	0	21	37	11
Image du back (<code>back:5bf1d6a2</code>)	12	0	5	7	0	0
Image du front (<code>front:5bf1d6a2</code>)	10	0	0	5	4	0
Total	80	0	5	33	41	11

Le total de 80 compte aussi un constat de sévérité inconnue sur l'image du front. Par nature : 58 défauts de configuration, 22 vulnérabilités, aucun secret. Les cinq HIGH sont des CVE de `jackson-core` et `jackson-databind` 2.21.4 dans l'image `back:5bf1d6a2`, qui n'est pas en service : la version forcée de Jackson les corrige, et le scan de cette image l'a empêchée d'être déployée. ⚠

Limite : l'index n'est pas réalimenté automatiquement ; il ne reflète pas encore les images 1.0.1 (§7.2).

4.4 Les exceptions : assumées, justifiées, datées

Une exception est une vulnérabilité signalée que l'on décide de ne pas corriger tout de suite. Elle n'est acceptable que justifiée et bornée dans le temps. Toutes celles du projet expirent le **31 décembre 2026** : passé cette date, le contrôle se referme et impose la relecture.

Fichier	Exceptions	Portée
<code>.trivyignore.yaml</code>	7 entrées, couvrant 11 constats	défauts de configuration de manifestes, justifiés un à un
<code>back/config/dependency-check/suppressions.xml</code>	3 entrées, couvrant 12 CVE Spring Framework	modules, fonctions ou conditions que l'application ne réunit pas

Les douze CVE de Spring Framework 6.2.19 méritent d'être détaillées, parce que **aucune montée de version gratuite ne les lève**. La version corrigée de la branche 6.2, la 6.2.20, est réservée au support payant de l'éditeur ; le seul correctif en source ouverte est Spring Framework 7, c'est-à-dire Spring Boot 4, une migration majeure. Attendre ne sert donc à rien.

- **Six CVE** visent WebFlux et RSocket, des modules absents du jar livré.
- **Trois CVE** visent des fonctions de Spring MVC que l'application n'utilise pas : elle ne déclare aucun contrôleur, seulement deux dépôts Spring Data REST.
- **Trois CVE** (CVE-2026-47886, CVE-2026-59282, CVE-2026-59283) touchent au langage d'expression SpEL et à la liaison de données. L'analyse, condition par condition contre les avis de l'éditeur, montre que pour chacune au moins une des conditions exigées manque. Elle est doublée d'un contrôle par exécution sur le back de staging : trois requêtes `PATCH` forgées ont chacune répondu `400` en moins de 0,1 s, sans effet sur la ressource.

⚠ **Ce que ces exceptions ne valent pas.** Elles ne prouvent pas que Spring 6.2.19 est sain : elles disent que les conditions d'exploitation ne sont pas réunies aujourd'hui. Elles tombent le jour où le modèle de données reçoit un champ `BigDecimal`, ou où du code évalue une expression venue d'une requête. La sortie propre est la migration vers Spring Boot 4. Le commentaire de `suppressions.xml` porte l'analyse complète.

La porte passe en CI. `dependency-check-back` est vert dans les trois pipelines de la release. Son rapport JSON, relu dans les artefacts, porte **82 dépendances analysées, aucune CVE ouverte de score ≥ 7 , et les 12 CVE Spring écartées par exception**.

5. Performance mesurée

5.1 Le budget de performance et les scénarios k6

Trois scénarios, écrits en JavaScript et versionnés à côté du reste, s'exécutent **après** l'étape `package` : GitLab démarre l'image Docker qui vient d'être construite comme un service, et k6 l'interroge. On mesure donc l'artefact qui partirait réellement en production, pas une compilation locale.

Scénario	Charge	Rôle	Bloquant
smoke.js	1 utilisateur, 5 parcours complets	« est-ce que ça marche ? »	oui
load.js	10 utilisateurs en lecture + 2 écritures/s	garde-fou des temps réponse	non
stress.js	paliers jusqu'à 50 utilisateurs, sans pause	trouver le point de rupture	déclenché à la main

Seuil	Valeur par défaut
p95 des lectures	500 ms
p95 des écritures	800 ms
Taux de requêtes en erreur	< 1 %
p95 du smoke (application froide)	1 500 ms

Deux choix structurent ces seuils. **Des percentiles et pas des moyennes** : une moyenne correcte peut très bien cacher 5 % d'utilisateurs qui attendent trois secondes. Et **un seuil par endpoint** plutôt qu'un seuil global, ce qui permet de dire « c'est la recherche par email qui décroche » au lieu de « c'est lent ». Chaque réponse est en outre vérifiée fonctionnellement, statut et contenu : un serveur qui renvoie 500 en 3 ms est très rapide et totalement cassé.

Les seuils vivent dans le dépôt et **ne sont pas exposés en variables GitLab** : un garde-fou dont on desserre le seuil depuis une interface pour faire passer un pipeline rouge n'est plus un garde-fou. Les variables CI règlent la charge, pas le budget.

⚠ Ces mesures dépendent de la machine. Elles servent à détecter une régression entre deux exécutions comparables, pas à annoncer une capacité absolue. La base étant une HSQLDB en mémoire, plus rapide qu'une base réseau, les chiffres sont optimistes en valeur absolue. Le front n'est pas testé en charge : seul le back l'est, parce que c'est lui qui porte le risque de saturation.

5.2 La latence du front, mesurée en service

Le journal d'accès de Caddy est la seule source qui voit passer le trafic du front, et il porte les trois mesures d'un coup. Les logs applicatifs du back disent ce que l'application raconte, pas le temps qu'elle met.

Percentile	Latence mesurée (105 requêtes, vérification initiale)
p50	0,14 ms
p95	1,60 ms
p99	3,11 ms

Sur la durée, la mesure tient : sur 624 tranches de cinq minutes relevées en sept jours, le p95 médian est de **1,54 ms**, et 90 % des tranches sont sous 2,75 ms (k8s/elk/alerting/README.md). Douze tranches dépassent 100 ms ; les quatre qui comptent au moins 20 requêtes correspondent toutes à des moments où le poste était saturé.

⚠ **C'est la latence vue par le serveur web, pas par l'API.** Caddy sert le bundle Angular et /config.json ; les appels à l'API partent du navigateur vers un hôte distinct et ne passent pas par lui. Et l'essentiel de ce trafic est fait des sondes de Kubernetes, pas d'utilisateurs.

⚠ **Le front ne produit quasiment jamais de 4xx**, parce que toute route inconnue renvoie 200 avec index.html, à charge pour le routeur Angular de décider. Vérifié : 12 requêtes vers des chemins inexistants, 12 réponses 200. Les erreurs réelles se lisent côté back.

5.3 La latence de l'API

Le back embarque un agent OpenTelemetry qui trace chaque requête et l'envoie à Elastic APM, sans qu'une ligne du code Java ait changé. **Les traces arrivent de staging et de production** : le 5 octobre, après des requêtes envoyées aux deux API, l'index traces-apm* a reçu 534 documents service.environment=staging en 15 minutes, puis 187 production et 180 staging en 10 minutes ; les logs du back portent un trace.id. Ce trafic est provoqué, pas celui d'utilisateurs : il prouve que les traces arrivent, il ne fournit pas une latence de référence en service, qui n'est pas relevée.

Les chiffres de latence disponibles viennent d'un essai du 2 octobre : l'image instrumentée lancée dans un conteneur local, et 303 requêtes envoyées en 18 secondes.

Transaction	Requêtes	p50	p95	p99
GET /{repository}	200	4,20 ms	5,65 ms	8,60 ms
GET /{repository}/{id}	80	2,64 ms	3,41 ms	6,03 ms
POST /{repository}	20	3,01 ms	5,02 ms	24,25 ms

{repository} est la route générique de Spring Data REST : /persons et /organizations y sont confondus.

Observability

Overview

Alerts

SLOs

Cases

AI Assistant

Logs

Explorer

Logs Anomalies

Logs Categories

Settings

Infrastructure

Infrastructure inventory

Metrics Explorer

Hosts

Applications

[Service Inventory](#)

Traces

Dependencies

Synthetics

Monitors

TLS Certificates

User Experience

Dashboard

Services

Tell us what you think!

Environment All

All services

Service groups

Inventory

Service Map

Search transactions, errors and metrics (E.g. transaction.duration.us > 300000 AND http.response.status_code >= 400)

Last 24 hours Refresh

☒ Comparison Day before

Search services by name

Name	Environment	Latency (avg.)	Throughput	Failed transaction rate
microcrm	verification-poste	4.6 ms	0.2 tpm	0%

Rows per page: 25

1

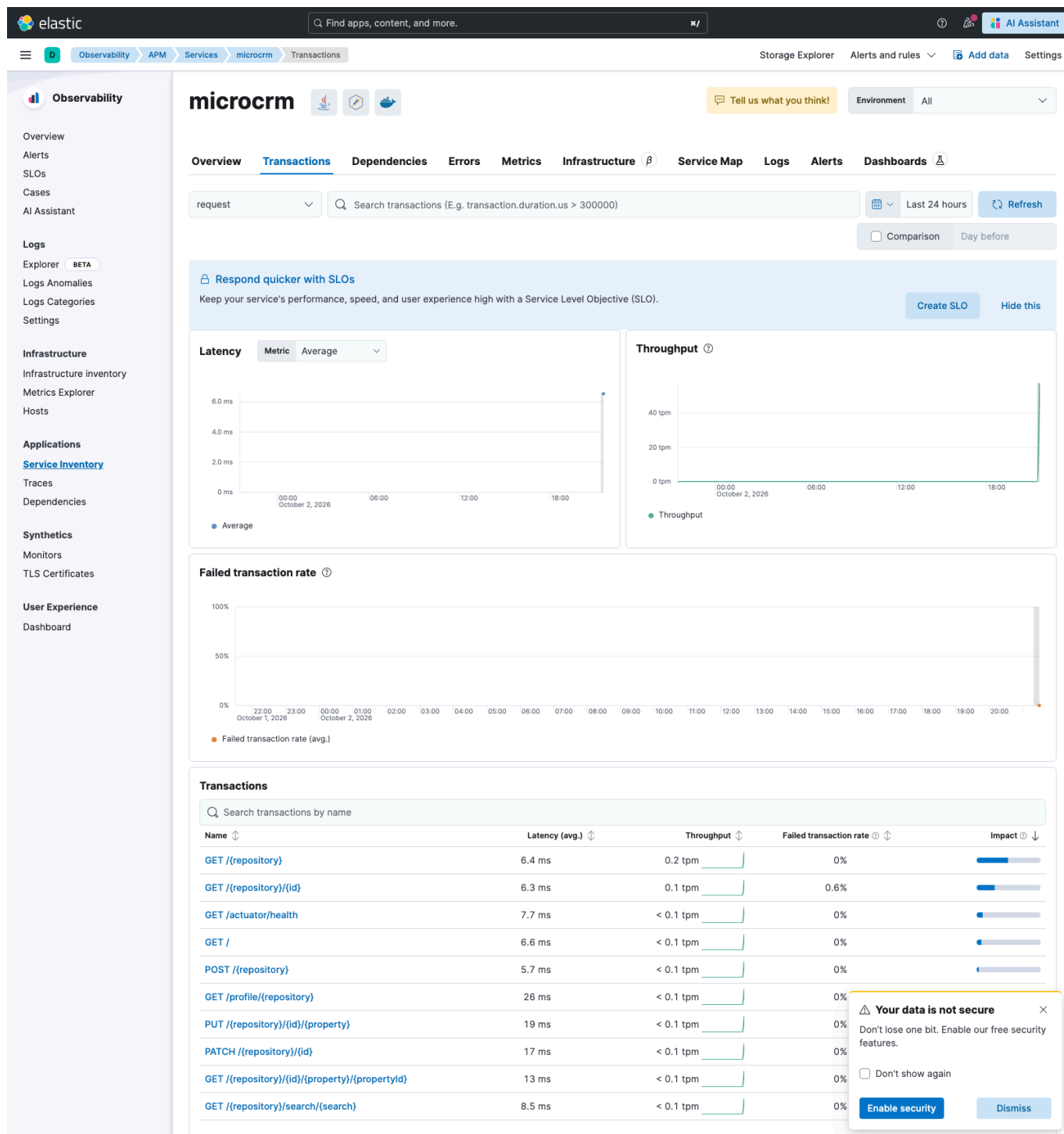
Your data is not secure

Don't lose one bit. Enable our free security features.

☐ Don't show again

Enable security

Dismiss



Ce qu'il faut y lire : l'étiquette d'environnement « verification-poste » sur la première capture. Elle dit que ces mesures viennent d'un essai lancé depuis le poste, pas de staging. Sur la seconde, toute l'activité tient dans la dernière heure : c'est un essai, pas un trafic.

⚠ **Trois réserves sur ces chiffres.** Ce sont les latences d'un conteneur local, sur un poste chargé, avec une base de démonstration presque vide : elles prouvent que la mesure existe, pas ce que vaut l'API de staging. La seconde capture agrège aussi les requêtes d'un autre essai local, celui des alertes (environnement `demo-alerting`) : ses chiffres diffèrent donc du tableau. Enfin l'agent a un coût, mesuré une seule fois : 408 Mio de mémoire contre 292 Mio, et un démarrage en 3,39 s contre 2,45 s. C'est un ordre de grandeur, à confronter à la limite de 768 Mio du pod : déployé, le pod n'a pas été tué par manque de mémoire, mais sa consommation réelle n'est pas mesurée.

Ce qui reste non mesuré : le CPU et la mémoire. Les traces disent ce que l'API fait de ses requêtes, pas ce que ses pods consomment. `kubectl top` répond `Metrics API not available`, faute de `metrics-server` (§6.7).

5.4 Comportement sous déploiement et sous panne

Deux campagnes sur cluster ont mesuré ce qu'un utilisateur subit pendant les opérations, en interrogeant l'API en continu à travers l'Ingress.

Situation	Mesure
Rollout initial du back	<code>rollout status</code> en 0, ~10,5 s
Déploiement + rollback (180 s d'observation)	1 615 requêtes, 1 611 en 200 — 99,75 %
Fenêtre d'indisponibilité pendant un déploiement	≤ 0,20 s, un seul échantillon en erreur
Suppression du pod back	16,1 s d'indisponibilité totale (98 × 503 sur 350 requêtes)
Démarrage applicatif du back	6,7 s à 9,8 s selon l'exécution (±45 %)
Délai jusqu'à <code>Ready</code>	10 s à 15 s ; 2 tentatives de <code>startupProbe</code> sur 30

Ces chiffres disent deux choses distinctes, qu'il ne faut pas confondre.

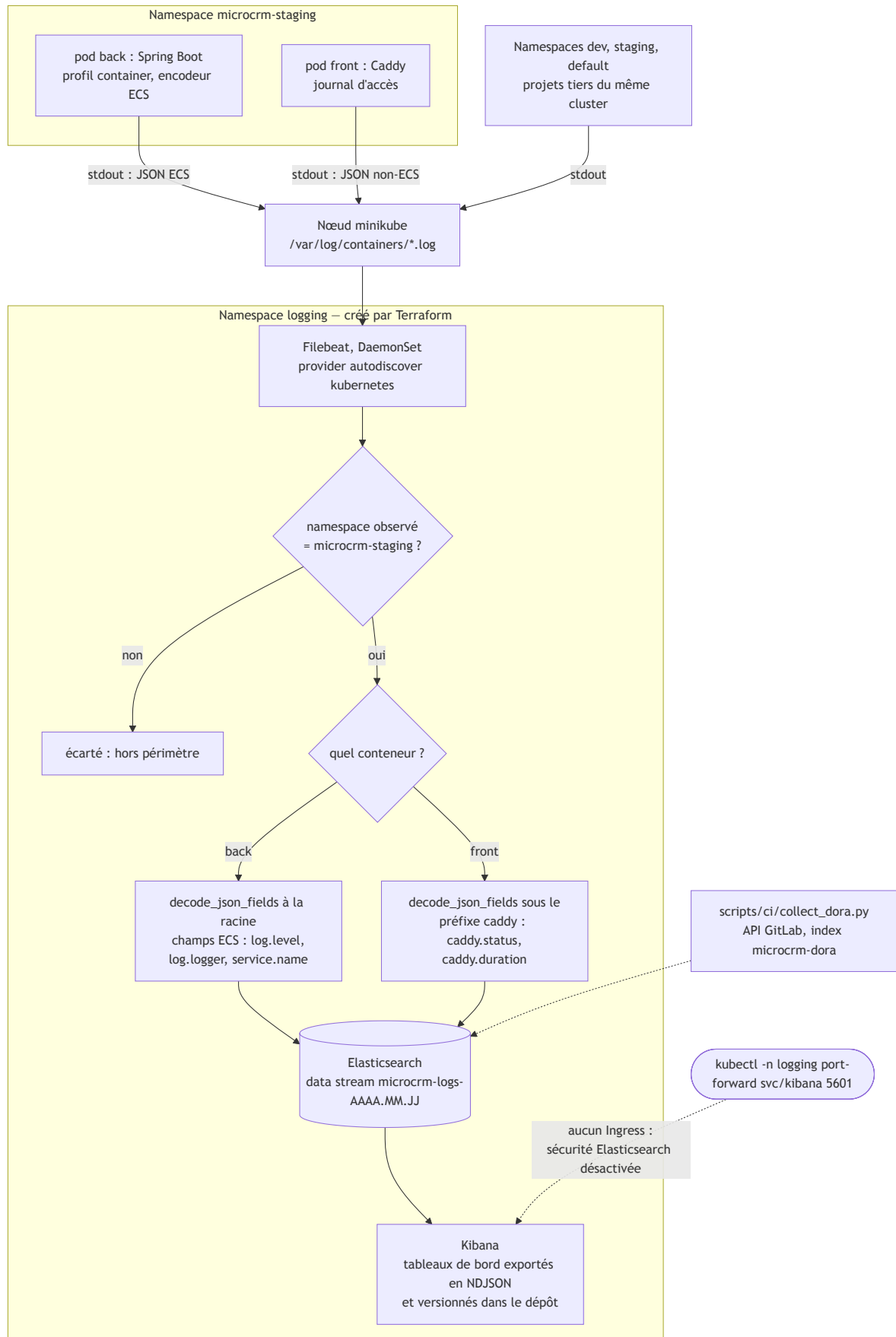
Un déploiement ne coupe pas. Le réglage `maxUnavailable: 0` impose que le nouveau pod soit prêt avant que l'ancien ne parte ; un déploiement vers une image inexistante ne touche pas le pod en service (`RESTARTS 0`, âge inchangé, API à 200 pendant toute la fenêtre).

Une perte de pod coupe seize secondes. C'est la conséquence directe du `replicas: 1` imposé par une base en mémoire : à un seul pod, il n'y a rien pour absorber la perte, et aucun `PodDisruptionBudget` ne protège d'une éviction. Le front, statique et à deux replicas en production, n'est pas affecté. La levée de cette limite passe par la sortie de HSQLDB, pas par un réglage.

La variance du démarrage (6,7 s à 9,8 s sur une machine identique et non chargée) justifie le choix d'un `startupProbe` plutôt que d'un `initialDelaySeconds` fixe : si le délai bouge de moitié sans qu'aucune variable ne change, un délai figé serait le mauvais outil. Le budget actuel — 150 s — est très surdimensionné (2 tentatives consommées sur 30) ; son vrai prix est l'autre bout, une JVM réellement bloquée occupant un slot 150 s avant d'être abandonnée.

6. Supervision : tableaux de bord, indicateurs et alertes

6.1 Le flux des logs



Source versionnée : `docs/schemas/flux-logs.mmd`, reprise dans `ARCHITECTURE.md §8.3`.

Le chemin lui-même est banal — un pod écrit sur `stdout`, le runtime en fait un fichier sur le nœud, un agent le lit. **Les deux losanges sont l'intérêt du schéma**, parce qu'ils portent les deux décisions sans lesquelles la chaîne ne tient pas.

Le premier filtre est une question de périmètre, pas de volume. Ce cluster n'est pas dédié à MicroCRM : il héberge les namespaces `dev` et `staging` d'autres projets, et quatre pods dans `default`. Un Filebeat non filtré y ingérerait les logs de tiers. Le filtre est posé deux fois — sur ce que l'autodiscover *observe*, et en condition sur ce qu'il *traite* — pour que l'élargissement de l'un ne fasse pas tomber l'autre. Sa contrepartie est à connaître : **seul `microcrm-staging` est collecté, la production ne l'est pas.**

Le second embranchement évite une panne irréversible. Le back produit de l'ECS, décodé à la racine ; le front produit lui aussi du JSON, mais non-ECS, dont les champs portent les mêmes noms sans avoir la même forme. Un décodage uniforme ferait rejeter ses documents, et une collision de mapping ne se répare pas : une fois le champ typé dans l'index, aucun document contradictoire n'y entre plus.

6.2 Ce qui est vérifié

Vérification	Résultat
Rollout Elasticsearch / Kibana / Filebeat	les trois <code>Running</code>
Documents indexés (2 octobre 2026)	92 175 , un seul namespace
Provenance	100 % <code>microcrm-staging</code> — aucun projet voisin collecté
Champs ECS décodés	<code>log.level</code> , <code>log.logger</code> , <code>service.name</code> , <code>process.thread.name</code>
Métadonnées Kubernetes	<code>kubernetes.pod.name</code> , <code>kubernetes.namespace</code> , <code>container.image.name</code>
Réimportation des tableaux de bord	cinq fichiers, <code>success: true</code> sur chacun (§6.4)
APM Server	1/1 <code>Running</code> , version 8.19.7
Traces du cluster (5 octobre 2026)	reçues de staging et de production, logs du back corrélés <code>trace.id</code>

Le livrable n'est pas « des écrans dans Kibana », ce sont des fichiers. Les tableaux de bord sont exportés en NDJSON et les règles d'alerte en JSON, tous versionnés dans le dépôt. Un écran ou une règle qui n'existe que dans une instance disparaît avec elle — et celle-ci tourne sur un poste de développement. Chaque panneau chiffré a été confronté à la même agrégation jouée directement contre Elasticsearch : mêmes chiffres des deux côtés (`k8s/elk/dashboards/README.md`).

Le dimensionnement de la stack est confronté à une charge réelle. Pendant le rollout de Kibana, le quota du namespace affichait `limits.memory 5376Mi/6Gi`, `requests.memory 3200Mi/4Gi`, `Pods 4/12` — **exactement les valeurs calculées dans le `terraform.tfvars`, au mégaoctet près**. La règle qui les gouverne : la limite mémoire vaut le double du tas, parce qu'un conteneur JVM dont la limite égale le tas est tué par l'OOM killer au premier pic hors-tas.

6.3 Les indicateurs retenus, et pourquoi

Un indicateur n'a de valeur que si l'on sait d'où il vient et ce qu'il ne dit pas. Les douze ci-dessous sont ceux que les tableaux de bord affichent.

Domaine	Indicateur (KPI)	Source	Pourquoi celui-là	Sa limite
Livraison	Les quatre indicateurs DORA	API GitLab, <code>collect_dora.py</code>	Mesurent le résultat de la chaîne, vitesse et stabilité ensemble	Très peu d'observations (§2.3)
Disponibilité	Sondes de Kubernetes servies par le front	journal d'accès de Caddy	Un battement régulier, 18 requêtes par minute : son silence est une panne	Ne distingue pas front arrêté et collecte arrêtée
Disponibilité	Démarrages du back	ligne <code>Started MicroCRMApplication</code>	Un back qui redémarre en boucle se voit là	Ce sont des démarrages, pas le compteur de redémarrages
Disponibilité	Requêtes terminées en erreur	logs du back, traces APM	Chaque occurrence est une réponse 500 servie à un utilisateur	Côté traces : peu de recul
Performance	Latence du front, p50 / p95 / p99	<code>caddy.duration</code>	Un percentile montre les utilisateurs lents qu'une moyenne cache	Latence du serveur de fichiers, pas de l'API

Domaine	Indicateur (KPI)	Source	Pourquoi celui-là	Sa limite
Performance	Latence et débit de l'API, par route	traces OpenTelemetry	Dit quelle route ralentit, pas seulement « c'est lent »	Chiffré sur un conteneur local ; en service sans recul
Performance	Volume de logs par conteneur	<code>microcrm-logs</code>	Révèle un pic d'activité ou un service devenu muet	Compte des lignes, pas des requêtes
Sécurité	Constats CRITICAL et HIGH ouverts	rapports Trivy, <code>microcrm-security</code>	Ce sont les deux sévérités qui arrêtent le pipeline	Vaut pour l'image publiée, pas pour le code pipeline
Sécurité	Constats ouverts dans le temps	idem	Montre si la dette de sécurité monte ou descend	Historique rejoué, pas vécu (§6.4)
Sécurité	Exceptions et jours avant échéance	<code>.trivyignore.yaml</code>	Une exception oubliée devient une faille acceptée pour toujours	N'inclut pas les exceptions Dependency-Check
Sécurité	Chemins sensibles demandés au front	<code>caddy.request.uri</code>	<code>/.env</code> , <code>/.git</code> : la signature d'un balayage automatique	Signale une tentative, pas une fuite
Alertes	Déclenchements par famille et par règle	index <code>microcrm-alerts</code>	Dit ce qui a sonné, quand, et si c'est rétabli	Un écran vide ne veut pas dire que tout va bien

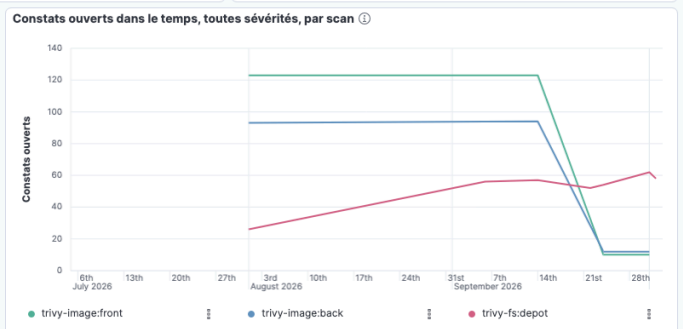
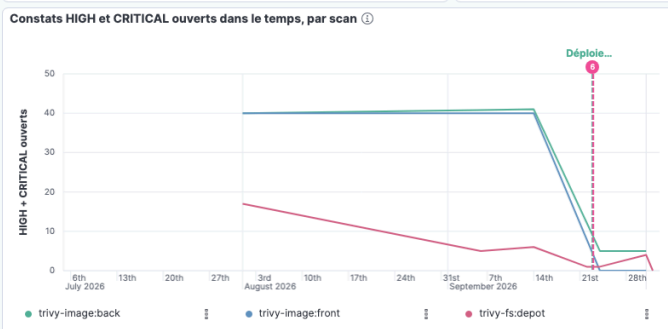
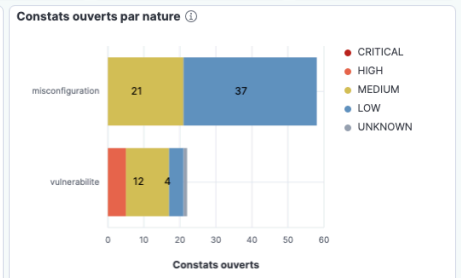
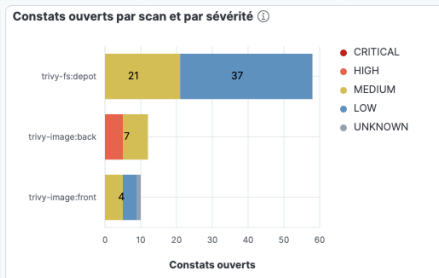
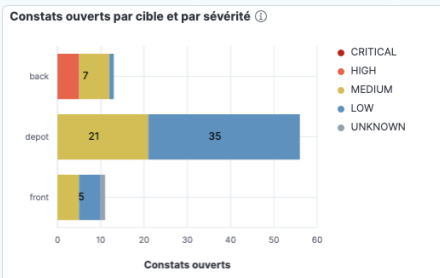
6.4 Les tableaux de bord

Cinq tableaux de bord sont versionnés dans `k8s/elk/dashboards/`. Tous ont été réimportés sur une instance d'où leurs objets avaient été supprimés.

Tableau de bord	Fichier	Objets	Ce qu'il suit	Capture
Supervision	microcrm.ndjson	8	performance et erreurs : volume, latence, statuts	§6.4
Sécurité	securite.ndjson	22	vulnérabilités, exceptions, erreurs	§6.4
Disponibilité	disponibilite.ndjson	18	présence, taux de réussite, démarrages, API	§6.4
Suivi des alertes	alertes.ndjson	8	déclenchements et rétablissements des huit règles	§6.5
Métriques DORA	dora.ndjson	13	les quatre indicateurs de livraison	§2.3

Les captures ci-dessous datent du 2 octobre 2026, celle des métriques DORA du 5 octobre.

⚠ **Limites.** Les scans antérieurs à aujourd'hui ont été rejoués sur d'anciens commits avec la base de vulnérabilités **d'aujourd'hui** : la courbe montre ce que ces versions contiennent de vulnérable au regard de ce qu'on sait maintenant, pas ce que le pipeline voyait à l'époque. Les panneaux HTTP ne voient que le **front** en staging : le back ne journalise pas ses réponses.



Vulnérabilités et constats HIGH / CRITICAL ouverts — dernier scan de chaque source ⓘ

5 documents										
	@timestamp	severity	identifiant	paquet	version	version_corrigee	composant	source	cible	commit
☒	Oct 1, 2026 @ 19:05:51.000	HIGH	CVE-2026-91777	com.fasterxml.jackson-core:jackson-databind	2.21.4	2.21.7, 2.18.11, 2.22.3	back	trivy-image:back	Java	5bf1d6a2
☐	Oct 1, 2026 @ 19:05:51.000	HIGH	CVE-2026-91776	com.fasterxml.jackson-core:jackson-databind	2.21.4	2.18.11, 2.21.7, 2.22.3	back	trivy-image:back	Java	5bf1d6a2
☐	Oct 1, 2026 @ 19:05:51.000	HIGH	CVE-2026-89425	com.fasterxml.jackson-core:jackson-core	2.21.4	2.21.7, 2.22.3, 2.18.11	back	trivy-image:back	Java	5bf1d6a2
☒	Oct 1, 2026 @ 19:05:51.000	HIGH	CVE-2026-89407	com.fasterxml.jackson-core:jackson-core	2.21.4	2.18.11, 2.21.7, 2.22.3	back	trivy-image:back	Java	5bf1d6a2
☐	Oct 1, 2026 @ 19:05:51.000	HIGH	CVE-2026-68497	com.fasterxml.jackson-core:jackson-databind	2.21.4	2.18.10, 2.21.6, 2.22.2	back	trivy-image:back	Java	5bf1d6a2

Exceptions assumées et leur échéance

7 documents

@timestamp

identifiant

chemin

echeance

jours_restants

expire

constats_couvrets

Justification

☒	Oct 2, 2026 @ 21:32:14.000	KSV-0056	.gitlab/agents/microcm/rbac.yaml	Dec 31, 2026 @ 01:00:00.000		90	false	3 Droits d'écriture sur les ressources networking accordés à l'agent ...
☐	Oct 2, 2026 @ 21:32:14.000	KSV-0041	.gitlab/agents/microcm/rbac.yaml	Dec 31, 2026 @ 01:00:00.000		90	false	1 Gestion des Secrets accordée à l'agent GitLab, et la règle a ...
☒	Oct 2, 2026 @ 21:32:14.000	KSV-0014	k8s/elk/elasticsearch-deployment.yaml	Dec 31, 2026 @ 01:00:00.000		90	false	1 Écart assumé et déjà commenté dans le manifeste. L'entypoint...
☒	Oct 2, 2026 @ 21:32:14.000	KSV-0014	k8s/overlays/production/back-resources-patch.yaml	Dec 31, 2026 @ 01:00:00.000		90	false	1 Faux positif de méthode, pas d'appréciation. Ce fichier est un patch ...
☒	Oct 2, 2026 @ 21:32:14.000	KSV-0118	k8s/overlays/production/b	Dec 31, 2026 @		90	false	2 Même cause que l'entrée



Ce qu'il faut y lire, de haut en bas. La tuile « 3 scans » garantit que les zéros voisins sont mesurés et non vides. La tuile rouge affiche 0 CRITICAL, la tuile orange 5 HIGH. La courbe de gauche montre la chute des constats graves sur les deux images. La première table nomme les 5 CVE et la version qui les corrige. La seconde liste les exceptions et leurs 90 jours restants.

Analyse. Trois enseignements.

- **Les constats graves sont divisés par huit sur l'image du back** (40, puis 5) et ramenés à zéro sur celle du front (40, puis 0). La bascule correspond au passage des scans Trivy en mode bloquant, accompagné de la montée de Spring Boot de 3.2.5 à 3.5.16 et de la recompilation de Caddy. Les 5 HIGH restants sont ceux de l'image `back:5bf1d6a2`, que la porte a refusée (§4.3).
- **Le dépôt passe de 17 constats graves à 0**, les défauts de configuration restants étant soit corrigés, soit couverts par une exception datée.
- **Ce qui reste ouvert est de faible sévérité** : 33 MEDIUM et 41 LOW, dont 58 défauts de configuration. Ils ne bloquent pas la livraison et forment la dette à résorber (§7.4).

⚠ **Trois réserves, sans lesquelles ces courbes se lisent mal.**

- **L'historique est rejoué, il n'a pas été vécu.** Huit anciens commits ont été scannés le 2 octobre, avec la base de vulnérabilités de ce jour-là, et datés de leur commit. La courbe dit « ce que ces versions contiennent de vulnérable au regard de ce qu'on sait aujourd'hui », pas « ce que le pipeline voyait à l'époque ».
- **Les trois panneaux du bas sont vides par construction.** Le front répond 200 à tout chemin, et le back, seul à pouvoir répondre 401 ou 403, ne journalise pas ses accès. Leur titre le dit. L'application n'a d'ailleurs pas d'authentification.
- **Dependency-Check n'y figure pas.** Le collecteur sait lire son rapport — testé sur un rapport fabriqué — mais le rapport réel, disponible en artefact, ne lui a pas été passé.

Disponibilité

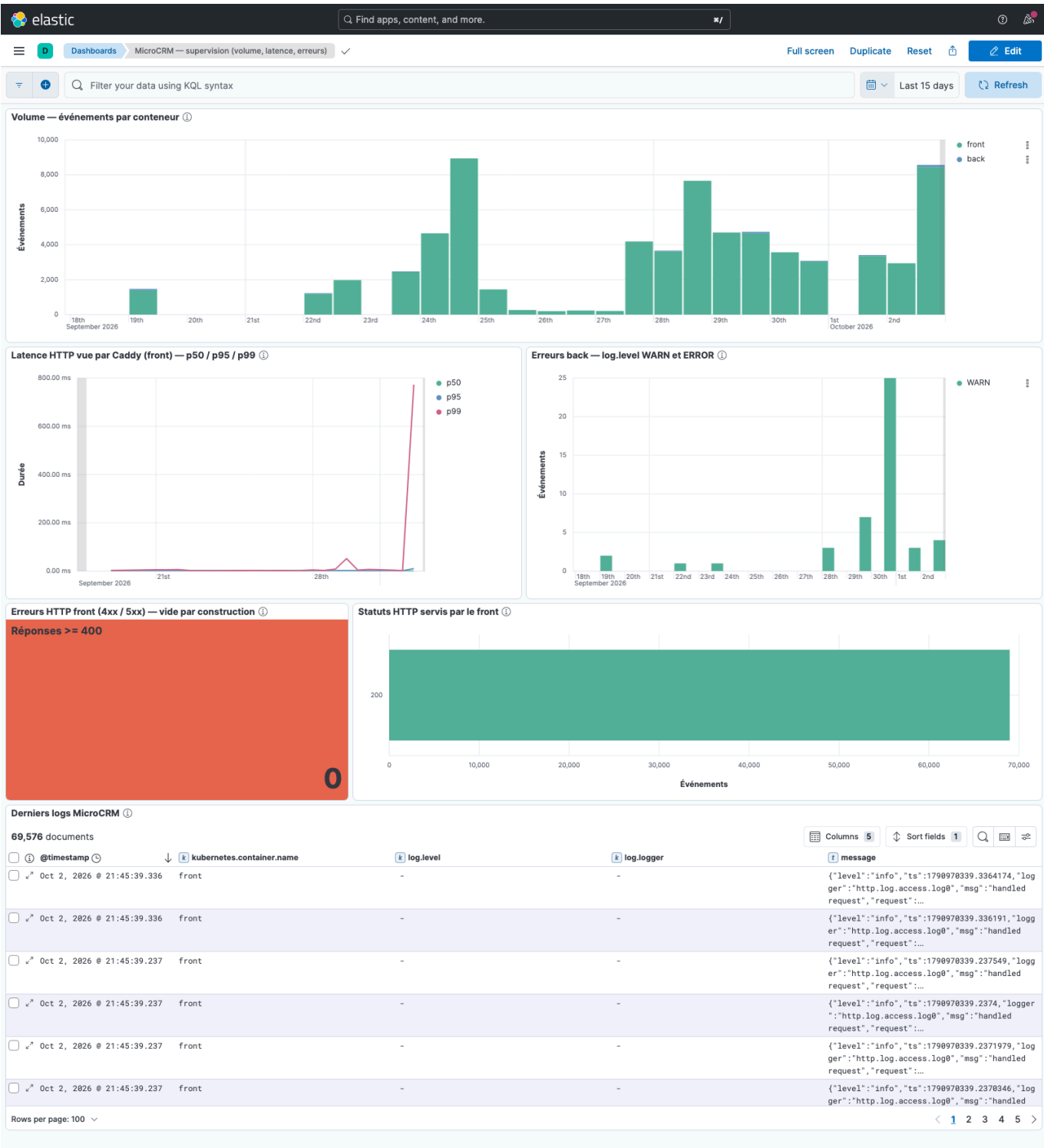


Ce qu'il faut y lire : la grande courbe verte est le battement de cœur du front. Ses trous sont les périodes où le poste était éteint. Les traits verticaux sont des annotations automatiques : un démarrage du back, un déploiement, un rollback.

Analyse. Ce tableau de bord est le plus facile à sur-lire, et il faut le dire nettement : **il ne mesure pas un taux de disponibilité, il montre une présence de logs.**

- **Le 100 % de réussite est vrai et ne prouve presque rien.** Le front répond 200 à tout, et 94 % de ses requêtes sont des sondes. Ce taux dit que Caddy répond.
- **Sur les 360 heures de la fenêtre, 189 portent au moins une sonde,** soit 52,5 %. Ce n'est pas un taux de disponibilité : le cluster est un minikube de poste, éteint la nuit, et un trou ne distingue pas « front arrêté » de « collecte arrêtée ».
- **Onze démarrages du back en quinze jours,** dont trois le 2 octobre. Ces trois-là s'expliquent par la relance du cluster et les essais d'alerte, pas par des pannes — c'est ce que les annotations permettent de vérifier d'un coup d'œil.
- **Les panneaux APM de cette capture viennent des deux essais locaux du 2 octobre.** Staging et production y alimentent depuis leurs propres transactions ; le tableau n'a pas été recapturé.
- **La production n'apparaît pas.** Le panneau « par environnement » n'a qu'une série, et il existe pour que cette absence se voie.

Performance



Ce qu'il faut y lire : la courbe de latence, en haut à gauche, reste plate pendant quinze jours puis monte brutalement à la dernière heure. Le panneau voisin compte les avertissements du back, jour par jour.

Analyse.

- **La pointe de latence du 2 octobre est provoquée, pas subie.** C'est l'essai de l'alerte de performance : le front de staging a été privé de CPU, sous charge, pour faire sonner la règle (\$6.5). Le reste de la fenêtre est plat, conforme au p95 médian de 1,54 ms.

- **Le back n'a journalisé aucune erreur.** Sur 90 jours : 52 lignes `WARN` , 0 `ERROR` . Les avertissements du back sont ceux qu'il écrit en démarrant, auxquels s'ajoutent ceux de l'essai d'alerte. La cause du pic de 25 lignes sur une seule journée n'est pas établie ligne à ligne.
- **Le volume suit les heures d'allumage du poste**, pas un trafic d'utilisateurs : il n'y a pas d'Ingress exposé, donc pas de visiteur.

6.5 Les alertes

Huit règles d'alerte Kibana couvrent les trois familles attendues. Ce sont des fichiers (`k8s/elk/alerting/rules/`), installés par `scripts/monitoring/install_alerting.py` , et non des réglages faits à la souris. Toutes s'évaluent chaque minute.

Famille	Règle	Ce qu'elle surveille	Seuil	Pourquoi ce seuil
Disponibilité	<code>dispo-front-muet</code>	le front de staging ne sert plus rien	moins de 1 requête en 3 min	54 sondes attendues en 3 min : zéro n'est jamais un creux, c'est une panne
Disponibilité	<code>dispo-back-redemarrages</code>	le back redémarre en boucle	2 démarrages ou plus en 15 min	12 démarrages en 47 jours, jamais plus d'un par quart d'heure sauf une fois
Disponibilité	<code>dispo-back-echecs-requetes</code>	des requêtes du back finissent en exception	1 ou plus en 5 min	aucune ligne de ce genre en 47 jours
Disponibilité	<code>dispo-api-5xx</code>	l'API répond en erreur 5xx (traces)	1 ou plus en 5 min	0 sur les 401 transactions relevées avant l'essai
Performance	<code>perf-front-p95</code>	le front ralentit	p95 > 100 ms sur 20 requêtes	65 fois le p95 médian de 1,54 ms ; le plancher écarte les tranches trop petites
Performance	<code>perf-api-p95</code>	l'API ralentit (traces)	p95 > 250 ms sur 20 transactions	2,5 fois le maximum observé (97 ms) ; échantillon petit, seuil à recalculer
Sécurité	<code>secu-front-chemins-sensibles</code>	quelqu'un demande <code>/.env</code> , <code>/.git</code> , <code>/wp-login.php</code> ...	1 requête ou plus en 5 min	sur 68 493 requêtes en 30 jours, le seul chemin demandé était <code>/</code>

Famille	Règle	Ce qu'elle surveille	Seuil	Pourquoi ce seuil
Sécurité	secu-api- rafale-4xx	énumération d'identifiants sur l'API (traces)	20 réponses 4xx ou plus en 5 min	trafic normal : 0 ; une énumération en produit 40 à 64

elastic

Q Find apps, content, and more.

⌵

ⓘ

🔔

Stack Management

Rules

Management

Ingest ⓘ
Ingest Pipelines

Data ⓘ
Index Management
Index Lifecycle Policies
Data Set Quality
Snapshot and Restore
Rollup Jobs
Transforms
Remote Clusters
Migrate

Alerts and Insights ⓘ
Alerts
Rules
Cases
Connectors
Reporting
Machine Learning
Maintenance Windows

Kibana ⓘ
Data Views
Files
Saved Objects
Tags
Search Sessions
Spaces
Advanced Settings

Stack ⓘ
License Management
Upgrade Assistant

Rules

Detect conditions using rules.

Rules

Logs

Q Search

Rule state ⓘ

Type ⓘ

Action type ⓘ

Last response ⓘ

Tags ⓘ

Refresh

● Succeeded: 8 ● Failed: 0 ● Warning: 0

Updated a few seconds ago

🔗 5 m

8 rules

Columns ⓘ 10

☐	Name ↑	Last run ⓘ	Notify ⓘ	Inter...	Duration ⓘ	P50 ⓘ	Success rati...	Last response	State	
☐	MicroCRM - disponibilité - back de staging qui redémarre en boucle Elasticsearch query	Oct 2, 2026 21:56:36pm a few seconds ago	🔔	1 min	00:00	00:01	100%	● Succeeded	Enabled ⓘ	...
☐	MicroCRM - disponibilité - front de staging muet Elasticsearch query	Oct 2, 2026 21:56:36pm a few seconds ago	🔔	1 min	00:00	00:00	100%	● Succeeded	Enabled ⓘ	...
☐	MicroCRM - disponibilité - requêtes du back de staging terminées en exception Elasticsearch query	Oct 2, 2026 21:56:33pm a few seconds ago	🔔	1 min	00:01	00:01	100%	● Succeeded	Enabled ⓘ	...
☐	MicroCRM - disponibilité - réponses 5xx de l'API (traces APM) Elasticsearch query	Oct 2, 2026 21:56:30pm a few seconds ago	🔔	1 min	00:01	00:01	100%	● Succeeded	Enabled ⓘ	...
☐	MicroCRM - performance - latence p95 de l'API (traces APM) Elasticsearch query	Oct 2, 2026 21:56:39pm a few seconds ago	🔔	1 min	00:00	00:00	100%	● Succeeded	Enabled ⓘ	...
☐	MicroCRM - performance - latence p95 du front de staging Elasticsearch query	Oct 2, 2026 21:56:42pm a few seconds ago	🔔	1 min	00:00	00:00	100%	● Succeeded	Enabled ⓘ	...
☐	MicroCRM - sécurité - chemins sensibles demandés au front de staging Elasticsearch query	Oct 2, 2026 21:56:48pm a few seconds ago	🔔	1 min	00:00	00:00	100%	● Succeeded	Enabled ⓘ	...
☐	MicroCRM - sécurité - rafale de réponses 4xx de l'API (traces APM) Elasticsearch query	Oct 2, 2026 21:56:42pm a few seconds ago	🔔	1 min	00:00	00:00	100%	● Succeeded	Enabled ⓘ	...

Rows per page: 10

< 1 >

⚠️ Your data is not secure

Don't lose one bit. Enable our free security features.

☐ Don't show again

Enable security

Dismiss

Ce qu'il faut y lire : les huit règles sont actives et leur dernière évaluation a réussi. « Succeeded » veut dire que la règle s'est exécutée sans erreur, pas qu'une alerte a sonné.

Preuve de déclenchement. Une règle qui n'a jamais sonné ne prouve rien. Le 2 octobre, entre 21 h 37 et 21 h 54 (heure de Paris), chacune des huit a été déclenchée volontairement, puis s'est rétablie seule.

Règle	Provoquée par	Déclenchée	Valeur relevée	Rétablie
dispo-back-echecs-requetes	une requête invalide sur le back de staging	21:37:35	1 exception	21:40:35
dispo-api-5xx	la même requête, sur un back instrumenté local	21:37:35	1 réponse 5xx	21:40:35
dispo-back-redemarrages	deux redémarrages du back de staging	21:38:38	3 démarrages	21:53:39
secu-api-rafale-4xx	60 demandes d'identifiants inexistants, back local	21:38:47	64 réponses	21:43:44
secu-front-chemins-sensibles	8 requêtes /.env, /.git/config ... au front	21:38:50	8 requêtes	21:43:50
perf-api-p95	back local bridé à 0,05 CPU	21:40:41	269,2 ms	21:44:41
perf-front-p95	front de staging bridé à 10 m CPU, sous charge	21:45:44	909,1 ms	21:51:45
dispo-front-muet	front de staging ramené à 0 replica	21:50:39	0 requête	21:51:39

dispo-back-redemarrages relève 3 et non 2 parce que le démarrage du cluster avait lui-même lancé le back douze minutes plus tôt.

elastic

Find apps, content, and more.

Full screen

Duplicate

Reset

Edit

MicroCRM — suivi des alertes (disponibilité, performance, sécurité)

Filter your data using KQL syntax

Last 1 hour

Refresh

Comment lire le suivi des alertes

Chaque ligne de ce tableau de bord est un changement d'état d'une règle d'alerte Kibana, écrit dans l'index `microcrm-alerts` par le connecteur `microcrm-alerts-index` : `declenchee` quand la condition devient vraie, `retablie` quand elle cesse de l'être. Une alerte qui dure dix minutes produit donc deux lignes, pas dix.

Les règles, leurs requêtes et leurs seuils sont des fichiers : `k8s/elk/alerting/rules/`, expliqués dans `k8s/elk/alerting/README.md`.

Un écran vide ne veut pas dire que tout va bien. Il veut dire qu'aucune règle n'a changé d'état sur la période — ce qui est aussi ce qu'on verrait si Kibana était arrêté ou si les règles avaient disparu. L'état des règles elles-mêmes se lit dans *Stack Management > Rules*, ou par `scripts/monitoring/install_alerting.py --etat`.

Ce que ces alertes ne voient pas : la production (Filebeat ne collecte que `microcrm-staging`), et l'API tant que le back déployé n'envoie pas de traces — les règles « traces APM » ne surveillent aujourd'hui que les environnements qui en émettent.

Déclenchements sur la période

Déclenchements

Chronologie des déclenchements, par famille

Famille	21:35	21:40	21:45	21:50
disponibilité	2	1	1	1
securite	1	0	0	0
performance	0	1	0	0

Déclenchements par famille

Famille	Déclenchements
disponibilité	4
performance	2
securite	2

Changements d'état par règle

Règle	declenchee	retablie
dispo-api-5xx	1	1
dispo-back-echecs-requetes	1	1
dispo-back-redemarrages	1	1
dispo-front-muet	1	1
perf-api-p95	1	1
perf-front-p95	1	1
secu-api-rafale-4xx	1	1
secu-front-chemins-sensibles	1	1

Journal des alertes

16 documents

@timestamp	famille	severite	regle_id	statut	valeur	unite	seuil	detail
Oct 2, 2026 @ 21:53:39.075	disponibilite	haute	dispo-back-redemarrages	retablie	-	démarrages	2 démarrages ou plus en 15 min	la condition n'est plus remplie
Oct 2, 2026 @ 21:51:45.017	performance	moyenne	perf-front-p95	retablie	-	ms	p95 > 100 ms sur 5 min (20 requêtes au moins)	la condition n'est plus remplie
Oct 2, 2026 @ 21:51:39.033	disponibilite	critique	dispo-front-muet	retablie	-	requêtes	moins de 1 requête servie en 3 min	la condition n'est plus remplie
Oct 2, 2026 @ 21:50:39.023	disponibilite	critique	dispo-front-muet	declenchee	0	requêtes	moins de 1 requête servie en 3 min	aucune ligne du journal d'accès Caddy : front arrêté, ou ...
Oct 2, 2026 @ 21:45:44.886	performance	moyenne	perf-front-p95	declenchee	989.1	ms	p95 > 100 ms sur 5 min (20 requêtes au moins)	p95 calculé sur 989 requêtes
Oct 2, 2026 @ 21:44:41.862	performance	moyenne	perf-api-p95	retablie	-	ms	p95 > 250 ms sur 5 min (20 transactions au moins), par ...	la condition n'est plus remplie
Oct 2, 2026 @ 21:43:50.851	securite	haute	secu-front-chemins-sensibles	retablie	-	requêtes	1 requête ou plus vers un chemin sensible en 5 min	la condition n'est plus remplie
Oct 2, 2026 @ 21:43:44.845	securite	moyenne	secu-api-rafale-4xx	retablie	-	réponses 4xx	20 réponses 4xx ou plus en 5 min, par environnement	la condition n'est plus remplie
Oct 2, 2026 @ 21:40:41.248	performance	moyenne	perf-api-p95	declenchee	269.2	ms	p95 > 250 ms sur 5 min (20 transactions au moins), par ...	environnement demo-alerting, p95 calculé sur 553 transactions
Oct 2, 2026 @ 21:40:35.239	disponibilite	haute	dispo-api-5xx	retablie	-	réponses 5xx	1 réponse 5xx ou plus en 5 min, par environnement	la condition n'est plus remplie
Oct 2, 2026 @ 21:40:35.233	disponibilite	haute	dispo-back-echecs-requetes	retablie	-	exceptions	1 exception journalisée ou plus en 5 min	la condition n'est plus remplie

Rows per page: 100

Your data is not secure

Don't lose one bit. Enable our free security features.

☐ Don't show again

Enable security

Dismiss

Ce qu'il faut y lire : chaque règle apparaît deux fois, « déclenchée » puis « retablie ». Le journal du bas donne la valeur qui a fait sonner — 909,1 ms pour la latence du front, 269,2 ms pour celle de l'API.

Un défaut applicatif rendu visible. La requête `GET /persons/abc` renvoie une erreur 500 au lieu d'un 400 : l'application traite une saisie invalide comme une panne. C'est un défaut à corriger côté code.

⚠ Ce qu'il ne faut pas sur-lire.

- **Aucune notification ne sort de Kibana.** Une alerte écrit une ligne dans un index et une ligne dans le journal de Kibana : il faut ouvrir le tableau de bord pour la voir. Les connecteurs courriel, Slack et webhook exigent une licence payante.
- **Les trois règles fondées sur les traces surveillent de vrais pods, mais n'ont sonné que sur un conteneur local.** Leurs requêtes regroupent par `service.environment` et couvrent donc staging comme production ; elles n'ont pas été re-déclenchées sur les pods déployés.
- **Les cinq autres ne voient que staging,** seul environnement dont Filebeat collecte les logs.
- **L'alerting ne se surveille pas lui-même.** Si Kibana tombe, aucune règle ne s'évalue, et un écran vide se lit alors comme « tout va bien ».
- **Un back arrêté sans redémarrer n'est détecté par aucune règle.**

6.6 Ce que les données apprennent : points de vigilance et actions

Constat tiré des tableaux de bord	Ce qu'il signifie	Action proposée (§7.4)
3 déploiements sur 15 échoués sur un cluster arrêté	La plateforme fait échouer des livraisons saines	Vérifier le cluster avant de déployer ; cluster dédié
5 CVE hautes sur une image refusée du back	<code>back:5bf1d6a2</code> , jamais en service, reste au registry	La supprimer ; réalimenter l'index de sécurité
Une seule série dans « présence par environnement »	Les logs de la production ne sont pas collectés	Étendre la collecte à la production
52,5 % des heures seulement portent des sondes	La disponibilité n'est pas mesurable sur un poste éteint la nuit	Une sonde externe, sur un cluster permanent
Les panneaux 401/403 et 4xx sont vides par construction	Les logs ne voient pas les accès à l'API ; les traces, si	Construire ces panneaux sur <code>traces-apm*</code>
16,1 s de coupure à la perte d'un pod	Un seul replica, imposé par la base en mémoire	PostgreSQL, puis deux replicas

Constat tiré des tableaux de bord	Ce qu'il signifie	Action proposée (§7.4)
GET /persons/abc répond 500	Une saisie invalide est comptée comme une panne	Corriger la conversion d'identifiant dans l'application
Mémoire avec agent : 408 Mio pour une limite de 768 Mio	La marge du pod est réduite, sur une seule mesure	metrics-server, puis recalage des resources

6.7 Ce que la supervision ne fait pas

Volet	État
Sondes de santé (Actuator)	fait, vérifiées sous kubelet ; /actuator/health à UP en production 1.0.1
Centralisation des logs	fait, pour staging seulement
Tableaux de bord versionnés	fait : cinq fichiers
Alerting	fait : 8 règles déclenchées et rétablies ; aucune notification hors de Kibana
Latence et débit de l'API	fait : traces reçues de staging et de production, sans recul (§5.3)
Métriques CPU et mémoire	absentes — pas de metrics-server, métriques JVM de l'agent coupées
Supervision de la production	partielle — traces oui ; logs non, Filebeat ne collecte que microcrm-staging
Disponibilité du cluster	non surveillée — un minikube arrêté ne se découvre qu'à l'échec d'un déploiement
Rétention des logs	absente — sans ILM, les données s'accumulent jusqu'au PVC de 5 Gio
Sécurité d'Elasticsearch	désactivée (xpack.security.enabled: false)
Exposition de Kibana	aucun Ingress ; accès par port-forward uniquement
Automatisation	aucun job de CI ne déploie, ne teste ni n'alimente cette stack

L'absence de métriques de ressources a une conséquence directe : **il manque jusqu'au signal sur lequel un autoscaler déciderait**, et les valeurs de resources des manifestes restent des estimations. Aucun OOMKill n'a été observé, ce qui est un indice, pas une mesure.

La sécurité désactivée d'Elasticsearch est assumée pour une stack locale de démonstration ; elle serait inacceptable ailleurs, et c'est la première chose à reprendre si cette stack devait sortir du poste. Les captures du §6.5 en portent la trace : Kibana y affiche lui-même « Your data is not secure ». C'est aussi pourquoi il n'y a pas d'Ingress : exposer une console d'administration sans authentification serait une mauvaise idée.

7. Gains obtenus et recommandations d'amélioration continue

7.1 Les indicateurs, avant et après

« Avant » désigne l'état initial du dépôt ou, pour les indicateurs de livraison, la période qui précède le premier déploiement réussi depuis la CI (jusqu'au 21 septembre 2026). « Après » désigne l'état livré, version 1.0.1. Le premier tableau compare les indicateurs mesurés ; le second, les propriétés de la chaîne.

Indicateur	Avant	Après	Réserve
Déploiements réussis depuis la CI	0 sur 7 tentatives	8 sur 15 (fenêtre de 30 jours)	sur trois journées
Fréquence de déploiement	0,0 par jour	0,2667 par jour (0,1667 au 23 septembre)	retombe à 0 sans nouveau déploiement
Délai de mise en production	non mesurable	4,76 h (médiane ; 1,38 h au 23 septembre)	contient l'attente du clic et un week-end
Temps de rétablissement	non mesurable	2,21 h (médiane ; 2,14 h au 23 septembre)	4 observations
Taux d'échec des changements	100 %	60 % (66,67 % au 23 septembre)	dont 3 échecs sur un cluster arrêté
Versions publiées	0	1.0.1, promue sans reconstruction, en production	une seule release
Contrôles non bloquants	6 jobs en <code>allow_failure</code> , Trivy en <code>--exit-code 0</code>	1 seul contrôle non bloquant (<code>k6-load</code>)	—
Dépendances Java analysées	0	82	—
Constats graves, image du back	40	0 sur <code>back:1.0.1</code> , en service	historique rejoué
Constats graves, image du front	40	0	historique rejoué

Indicateur	Avant	Après	Réserve
Constats graves, dépôt	17	0	historique rejoué
Couverture des branches du back	62,5 %	100 %	—
Tests du front	73	112	compte local ; test-front vert en CI
Assertions sur les scripts	266	430	—
Assertions sur les manifestes	108	151	—
Latence de l'API	aucune mesure	p50, p95, p99 par route ; traces en service	chiffres d'un conteneur local, pas du cluster
Règles d'alerte	0	8, toutes déclenchées puis rétablies	sans notification externe
Tableaux de bord versionnés	0	5 (69 objets)	—
Contexte de build du front	1 195 Mo envoyés au démon Docker	0,6 Mo	—
Contexte de build du back	51 Mo	0,1 Mo	—

Sujet	Avant	Après
Images d'outillage du pipeline	9 images flottantes ou en latest	toutes figées, déclarées une seule fois
Chaîne de compilation du back	trois JDK pour un même artefact (17 / 21 / 21)	une seule version, alignée Dockerfile ↔ pipeline
URL de l'API du front	compilée dans le bundle → localhost:8080 en prod	lue au démarrage depuis /config.json — une image pour tous les environnements
Utilisateur des conteneurs	root , aucun utilisateur déclaré	UID 1000 des deux côtés, aligné sur les manifestes
Port déclaré par le back	EXPOSE 4200 pour une application sur 8080	8080

Sujet	Avant	Après
Création d'un environnement	<code>kubectl create namespace</code> tapé par quelqu'un	décrit en Terraform, avec quota, limites et politiques
Prérequis du poste	deux phrases dans deux documents	playbook Ansible idempotent (<code>ok=23 changed=0</code>)
Capacité de retour arrière	<code>rollback-production</code> ramenait un <i>placeholder</i> inexistant	ramène la version précédente réellement déployée, vérifié sur cluster
Reconstruction d'un environnement	une procédure écrite	jouée le 22 septembre depuis une destruction réelle (<code>RELEASE.md §9.5</code>)
Lecture des logs	<code>kubectl logs</code> , un pod à la fois, sans historique	data stream Elasticsearch, cinq tableaux de bord versionnés
Détection d'un incident	quelqu'un regarde un écran, ou personne	8 règles évaluées chaque minute
Mesure de la livraison	aucune	4 indicateurs DORA calculés sur 67 pipelines, tous renseignés, et un job en CI
Release	aucune	tag <code>v1.0.1</code> → images promues à l'identique → Release GitLab créée par la CI

Trois de ces gains valent plus que les autres, parce qu'ils changent une propriété et pas seulement un chiffre.

« **Construire une fois, déployer partout** » est vrai. Caddy fabrique un `/config.json` à partir de son environnement et le bundle le lit avant de démarrer : l'URL de l'API n'est plus compilée dans l'image. La même image, construite et scannée une fois, passe de staging à production sans être reconstruite. La release 1.0.1 le prouve : `back:1.0.1` et `back:08a216b0` ont la même empreinte (§7.3).

Le retour arrière vise une version réelle. Avec deux révisions par déploiement, `kubectl get deploy` affichait `READY 1/1` pendant que le rollback aurait visé une image inexistante — une supervision branchée sur ce résumé n'aurait levé aucune alerte. C'est le genre de défaut qu'aucune relecture de YAML ne trouve, et que seule une campagne sur cluster révèle.

La reconstruction d'un environnement est jouée, pas seulement écrite. Le namespace de staging a été détruit puis reconstruit depuis le seul dépôt : Ansible en `ok=23 changed=0` , six ressources recréées par Terraform, rollout des deux Deployments en 11 secondes, API de nouveau servie (`RELEASE.md §9.5`). La restauration de *données*, elle, n'a pas d'objet tant que la base vit en mémoire.

7.2 Les gains de sécurité

Sujet	Avant	Après	État de la preuve
Scans Trivy	aucun, puis informatifs (<code>--exit-code 0</code>)	bloquants sur HIGH et CRITICAL	en CI
Dependency-Check	aucun	analyse 82 dépendances, bloquant à CVSS ≥ 7	en CI : rapport relu dans les artefacts
Scan d'image	aucun	avant le push : une image refusée n'est jamais publiée	en CI : <code>package-*</code> verts dans trois pipelines
Rapports de scan	aucun	JSON et tableau lisible, en artefacts de <code>trivy-fs</code> et <code>package-*</code>	en CI, téléchargeables sans jeton
Vue d'ensemble des vulnérabilités	aucune	collecteur <code>collect_security.py</code> , index <code>microcrm-security</code> , tableau de bord	alimenté à la main le 2 octobre
Exceptions	aucune règle	7 exceptions Trivy et 3 Dependency-Check, toutes justifiées, échéance 31/12/2026	relues dans les deux fichiers
Supervision de la sécurité	aucune	2 règles d'alerte, déclenchées et rétablies	l'une sur staging, l'autre sur un conteneur local
CVE de Jackson et de Log4j	—	Jackson 2.21.7 et Log4j 2.25.5 forcés	0 constat sur <code>back:1.0.1</code> , en production

Ce que ces gains apportent à la fiabilité d'ensemble : **une faille connue est arrêtée au plus tôt, et de façon visible**. La chaîne s'est déjà arrêtée sur `trivy-fs` et sur le scan de l'image du back, pour des CVE réelles : c'est un échec de pipeline qui protège la production, même s'il ne flatte pas les indicateurs.

Recommandations pour renforcer le suivi et la correction des vulnérabilités, par ordre d'utilité :

1. **Brancher le collecteur de sécurité dans le pipeline.** L'index `microcrm-security` n'est alimenté qu'à la main : l'historique du tableau de bord ne grandit pas tout seul. Un job qui exécute `collect_security.py` après les scans le rendrait vivant.

2. **Collecter le rapport Dependency-Check réel.** Il existe dans les artefacts de `dependency-check-back`, mais n'est pas passé au collecteur ; tant que ce n'est pas fait, les douze exceptions Spring n'apparaissent dans aucun tableau de bord.
3. **Vérifier qu'un scan a lu quelque chose.** Un contrôle du nombre de dépendances analysées dans `dependency-check-back` distinguerait un scan vide d'un scan sain.
4. **Suivre l'échéance des exceptions.** Elles expirent toutes le 31 décembre 2026. La tuile « jours avant la première échéance » existe ; une alerte à 30 jours éviterait de le découvrir par un pipeline rouge.
5. **Migrer vers Spring Boot 4** avant cette date : c'est la seule sortie qui retire les douze exceptions au lieu de les renouveler.
6. **Ajouter npm audit à l'étape security**, seul angle mort de dépendances du projet.
7. **Supprimer du registry l'image back:5bf1d6a2**, refusée par le scan pour ses cinq CVE. Elle n'a jamais été en service, mais elle reste promouvable par erreur.
8. **Scanner périodiquement les images déjà déployées.** Une image acceptée un jour peut être déclarée vulnérable le lendemain : un scan planifié des images en service, indépendant des commits, comblerait cet écart.
9. **Activer la sécurité d'Elasticsearch**, sans quoi l'outil qui supervise la sécurité est lui-même ouvert.

7.3 Observations sur la phase de release

Une release, ici, consiste à poser un tag de version sur un commit de `main`. Le pipeline de tag ne reconstruit rien : il **promeut** les images déjà construites et scannées pour ce commit, en leur ajoutant le numéro de version, puis le job `release` crée la Release GitLab. La procédure est dans `RELEASE.md §7`.

Preuve : la release 1.0.1, jouée le 5 octobre 2026.

Pipeline	Référence	Résultat
#2909284076	<code>develop</code> , commit <code>5296658a</code>	vert sur tous ses jobs automatiques ; <code>deploy-staging</code> réussi
#2912362926	<code>main</code> , commit de fusion <code>08a216b0</code>	vert sur ses jobs automatiques, dont <code>quality-gate</code> ; <code>deploy-production</code> réussi (après un échec sur cluster arrêté)
#2913490784	tag <code>v1.0.1</code> sur <code>08a216b0</code>	aucun <code>package-*</code> ; <code>promote-back</code> , <code>promote-front</code> et <code>release</code> verts ; Release <code>v1.0.1</code> créée ; production en 1.0.1

La promotion sans reconstruction est prouvée par les empreintes. Lues dans le registry, `back:1.0.1` et `back:08a216b0` ont le même digest `sha256:6d62a515d6d7bc6d27af9b8c2a8e6ae58eeb816836724b9bac0b0ede6337f32f` ; `front:1.0.1` et `front:08a216b0` le même digest `sha256:74474e9b81ebc2c8241bf04d2095b0729de5e31b62418b8419edc0dbe5e0e99e`. Ce qui tourne

en production est donc, octet pour octet, ce que le pipeline de `main` a testé, scanné (0 HIGH ou CRITICAL) et mis sous charge. La production exécute `back:1.0.1` (1 replica) et `front:1.0.1` (2 replicas), `/actuator/health` répond UP, `GET /persons` 200, et la ConfigMap porte les clés OpenTelemetry.

Les raisons de choisir GitLab

Tarification

Explorer

Q Rechercher ou accéder à...

Essai gratuit

Connexion

Projet

G-rez-le-cycle-de-vie-de-d-veloppement-logiciel

Épinglé

Membres

Éléments de travail

Branches

Requêtes de fusion

Pipelines

Feature catalog

Gestion

Planification

Code

Compilation

Déploiement

Releases

Registre de paquets

Registre de conteneurs

Opération

Surveillance

Analyse

Ted Pasquiet / G-rez-le-cycle-de-vie-de-d-veloppement-logiciel / Releases / MicroCRM v1.0.1

MicroCRM v1.0.1

Créé il y a 2 heures

08a216b0

v1.0.1

Notes de version

MicroCRM 1.0.1

Commit : 08a216b0 Pipeline de release : https://gitlab.com/pasquietted/g-rez-le-cycle-de-vie-de-d-veloppement-logiciel/-/pipelines/2913490784

Images promues

Composant	Version	Image éprouvée (même digest)
back	registry.gitlab.com/pasquietted/g-rez-le-cycle-de-vie-de-d-veloppement-logiciel/back:1.0.1	registry.gitlab.com/pasquietted/g-rez-le-cycle-de-vie-de-d-veloppement-logiciel/back:08a216b0
front	registry.gitlab.com/pasquietted/g-rez-le-cycle-de-vie-de-d-veloppement-logiciel/front:1.0.1	registry.gitlab.com/pasquietted/g-rez-le-cycle-de-vie-de-d-veloppement-logiciel/front:08a216b0

Ces images n'ont pas été reconstruites pour la release : ce sont celles que le pipeline du commit a construites, scannées et mises sous charge, auxquelles le numéro de version a été ajouté (retag).

La mise en production reste une action manuelle : job `deploy-production` de ce pipeline.

Déploiements

Environnement	État	ID de déploiement	Validation	Déclenché par	Créé	Terminé
production	Échec	1603201061	Merge pull request... 08a216b0		Il y a 1 heure	Il y a 47 minutes
production	En attente	1603201066	Merge pull request... 08a216b0		Il y a 1 heure	
production	Échec	1603380447	Merge pull request... 08a216b0		Il y a 44 minutes	Il y a 43 minutes
production	Succès	1603406902	Merge pull request... 08a216b0		Il y a 34 minutes	Il y a 33 minutes

Ressources

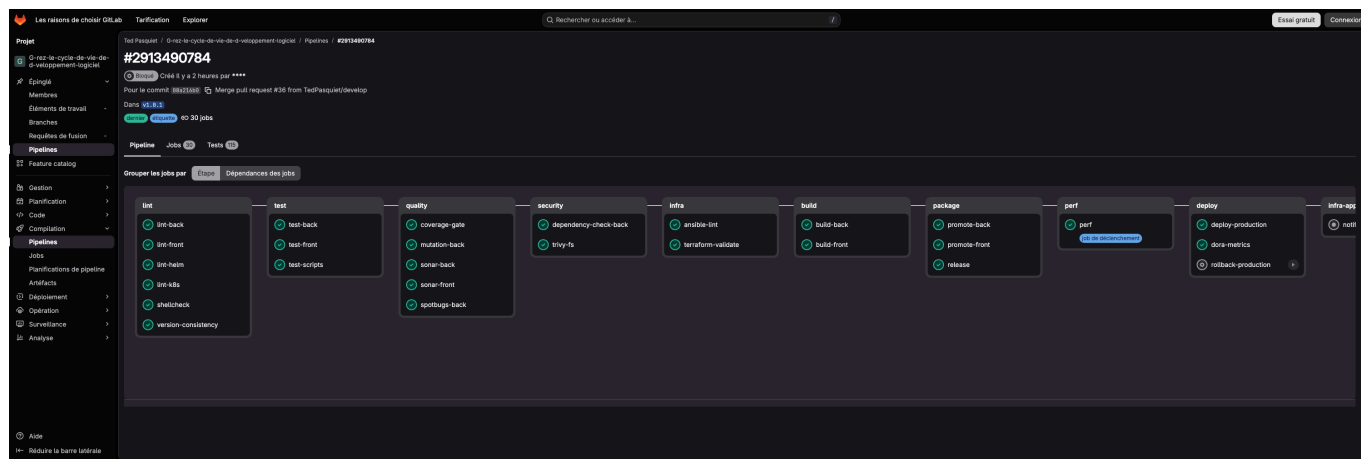
Collecte de preuves

v1.0.1-evidences-24785539.json

3d61e685

Collecté 1 hour ago

Ce qu'il faut y lire : la Release n'est pas rédigée à la main, c'est le job `release` qui l'écrit. Elle relie le numéro de version au commit, au pipeline et aux deux images. Les deux échecs de déploiement listés en bas sont ceux du cluster arrêté ; la ligne « En attente » est le job `rollback-production`, manuel, jamais lancé.



Ce qu'il faut y lire : l'étape `package` ne contient aucun `package-back` ni `package-front`. Sur un tag, le pipeline ne construit pas d'image : il ajoute un numéro de version à celles qui existent.

The screenshot shows the GitLab CI/CD interface for a project named 'G-rez-le-cycle-de-vie-de-d-veloppement-logiciel'. The 'Environnements' (Environments) page is active, showing a table of environments and their deployments.

Nom	Déploiements	Actions
logging	- En attente - Merge pull request #36 from TedPasquet/develop -> 08a216b8 - Action déclenchée par @project_84606666_bot_d7f5515a732a930cd17d52c01472b956 , le 5 octobre 2026 à 09:14:16 UTC+2	
production	- En attente - Merge pull request #36 from TedPasquet/develop -> 08a216b8 => v1.0.1 - Action déclenchée par @project_84606666_bot_d7f5515a732a930cd17d52c01472b956 , le 5 octobre 2026 à 14:47:35 UTC+2 - Succès - Merge pull request #36 from TedPasquet/develop -> 08a216b8 => v1.0.1 - Action déclenchée par @pasquietted , le 5 octobre 2026 à 16:11:13 UTC+2	
staging	- En attente - Merge pull request #36 from TedPasquet/develop -> 08a216b8 - Action déclenchée par @project_84606666_bot_d7f5515a732a930cd17d52c01472b956 , le 5 octobre 2026 à 09:14:16 UTC+2 - Succès - Merge pull request #35 from TedPasquet/docs/mise-a-jour-de... -> 5296658a - Action déclenchée par @pasquietted , le 5 octobre 2026 à 08:52:32 UTC+2	

Ce qu'il faut y lire : GitLab sait quelle version tourne dans chaque environnement. Les lignes « En attente » sont des jobs manuels jamais lancés : `terraform-apply-logging` et `terraform-apply-staging` du pipeline de `main`, `rollback-production` du pipeline de `tag`.

Observations issues de l'exécution du pipeline et des déploiements. Aucun incident n'est venu de l'application ni d'un contrôle de la chaîne. Tous viennent de la plateforme : Docker Desktop porte à la fois le runner et minikube, avec 8 Go de mémoire dont 6 réservés à minikube et partagés avec d'autres piles.

Observation	Cause	Recommandation (état)
Trois <code>deploy-production</code> en échec (38 à 48 s) sans rien écrire dans le namespace	cluster minikube arrêté, ou redémarré sans <code>minikube start</code> : l'agent GitLab journalise <code>dial tcp 10.96.0.1:443: i/o timeout</code>	Vérifier le cluster (<code>kubectl get --raw /readyz</code>) en tête des jobs de déploiement (à faire)
<code>develop</code> attend près d'une heure avant de démarrer ; un job reste bloqué 36 min (<code>runner_interrupted</code>)	runner unique à <code>concurrent = 1</code> , qui ne prend plus de jobs après un redémarrage de Docker	<code>concurrent > 1</code> ou un second runner (à faire)
<code>terraform-plan</code> échoue deux fois (~75 s), puis passe en 22 s une fois le cluster relancé	cluster figé	Une alerte sur la disponibilité du cluster, hors du cluster (à faire)
Un pipeline de tag peut échouer sur une image d'outillage résolue pour la mauvaise architecture	runner sur Apple Silicon : un tag d'image multi-architecture y résout vers arm64, sans Chrome	Images d'outillage désignées par leur empreinte amd64 (en place) ; tester le chemin de tag par un tag de pré-version (à faire)
Un tag ne dit pas quelles images portent la version	aucune trace écrite entre tag et images	Job <code>release</code> : Release GitLab avec commit, pipeline et images (en place)
Une image refusée par le scan pourrait être promue si elle était au registry	la promotion ne vérifie que l'existence du tag de commit	Scan avant le push (en place)
La version déclarée peut diverger entre fichiers	<code>package-lock.json</code> , <code>package.json</code> et <code>build.gradle</code> portent chacun la version	Contrôle <code>version-consistency</code> sur les tags (en place, 7 s)
Un pipeline de <code>main</code> sain s'affiche « Bloqué », ce qui se lit comme un échec	jobs manuels non <code>allow_failure</code> en attente (<code>terraform-apply-*</code> , <code>rollback-production</code>)	Expliqué dans la procédure et dans ce rapport (§2.3)
Le délai de mise en production mesure un clic, et un week-end	les deux déploiements sont <code>when: manual</code>	Déploiement de staging automatique, production manuelle (à faire, A3.1 du plan)

Ce que ces observations enseignent. La promotion est le bon mécanisme : l'image mise en production est exactement celle qui a été testée et scannée, et les empreintes le prouvent. Les contrôles tiennent. Ce qui lâche, c'est l'hôte : une chaîne entière — runner, cluster, supervision — qui dépend d'un seul Docker Desktop de poste n'est pas plus disponible que lui. La réponse de fond est un **cluster dédié**, hors du poste de développement : c'est la décision attendue du CTO.

7.4 Recommandations d'amélioration continue

Classées par ce qu'elles débloquent, et non par leur difficulté. L'effort est noté S (quelques heures), M (quelques jours).

Ce qui fiabilise la chaîne

Piste	Effort	Ce que ça change pour Orion
Vérifier le cluster (/readyz) en tête des jobs de déploiement	S	Un cluster arrêté est nommé tout de suite, au lieu d'échouer en <code>script_failure</code> anonyme.
<code>concurrent</code> > 1, ou un second runner	S	Un runner figé ne gèle plus toute la livraison.
Une alerte sur l'indisponibilité du cluster, hors du cluster	S	La panne de la plateforme se voit avant le déploiement, pas pendant.
Un cluster dédié, hors du poste de développement	M	Supprime la première cause d'échec de livraison mesurée.
Re-déclencher les trois règles APM sur les pods déployés	S	Elles cessent d'être prouvées seulement sur un conteneur local.
Recaler le seuil de <code>perf-api-p95</code> sur un trafic réel	S	Le seuil actuel repose sur 401 transactions d'un conteneur local.

Ce qui rend la mesure fiable

Piste	Effort	Ce que ça change pour Orion
<code>metrics-server</code> , puis des métriques de ressources	M	Donne CPU et mémoire — de quoi vérifier les <code>resources</code> , aujourd'hui estimées, et le surcoût de l'agent.
Étendre la collecte des logs à la production	S	La production cesse d'être invisible.
Une sonde externe de disponibilité	S	Remplace la « présence de logs » par un vrai taux de disponibilité.
Un pont entre <code>microcrm-alerts</code> et <code>notify.py</code>	S	Une alerte prévient quelqu'un. C'est un petit programme, sans licence payante.

Piste	Effort	Ce que ça change pour Orion
Alimenter les index DORA et sécurité depuis la CI	M	Les tableaux de bord cessent de dépendre d'un geste manuel. Suppose un Elasticsearch joignable par le runner.
Un runner dédié pour <code>k6-load</code>	M	Rend le seuil de charge défendable, donc bloquant.
<code>kubeconform</code> dans <code>lint-k8s</code>	S	Valide les manifestes contre le schéma de l'API, hors ligne.

Ce qui lève les limites structurelles

Piste	Effort	Ce que ça change pour Orion
Remplacer HSQLDB par PostgreSQL persistant	M	Lève deux limites d'un coup : le plafond à 1 replica et les 16 s de coupure sur perte de pod. Impose Flyway ou Liquibase.
Migrer vers Spring Boot 4	M	Retire les douze exceptions de sécurité avant leur échéance.
ILM sur les data streams et les index	S	Borne la croissance des logs, des alertes et des constats.
Activer la sécurité d'Elasticsearch	M	Condition pour que la stack sorte d'un poste de développement.
Un CNI qui implémente les NetworkPolicy	S	Le cloisonnement réseau cesse d'être décrit pour devenir effectif.
<code>jlink</code> sur l'image du back	M	390 Mo → ~150 Mo, en ne gardant que les modules réellement utilisés.

Le contexte d'Orion derrière cet ordre. Une petite équipe ne peut pas tout mener de front. La première série demande peu : elle fiabilise une chaîne qui livre déjà, et répond directement aux observations de la release 1.0.1 — sauf le cluster dédié, qui est une décision de moyens. La deuxième transforme une supervision de démonstration en outil d'exploitation. La troisième engage des choix d'architecture, et c'est elle qui appelle une décision du CTO.

8. Ce qui manque, et ce qui n'est pas prouvé

Cette liste est donnée sans enrobage : elle est ce qu'un jury est en droit d'attaquer, et il vaut mieux qu'elle vienne du rapport que de la lecture.

8.1 Ce qui n'est pas prouvé en conditions réelles

- **Les trois règles d'alerte fondées sur les traces n'ont sonné que sur un conteneur local.** Elles portent sur de vrais pods, mais n'y ont pas été re-déclenchées.

- **La latence de l'API en service n'est pas relevée.** Les traces arrivent ; le seul trafic reçu est celui de requêtes provoquées pour le vérifier.
- **Les journaux de jobs ne sont pas lisibles sans jeton.** Les causes d'échec citées viennent des statuts et motifs de l'API, du cluster et des artefacts ; les sorties des jobs eux-mêmes n'ont pas été relues.
- **L'historique du tableau de bord de sécurité est rejoué,** et le rapport Dependency-Check réel, disponible en artefact, n'y est pas collecté.
- **Le taux d'échec DORA mêle pannes de la plateforme et échecs de changement,** sans que le collecteur sache les séparer (§2.3).
- **La tenue des alertes dans la durée n'est pas connue :** aucun recul sur les faux positifs. L'une d'elles sonne au réveil d'un poste mis en veille.

8.2 Ce qui manque

- **Une plateforme fiable.** Le runner, le cluster et la supervision dépendent d'un seul Docker Desktop de poste. Aucune alerte ne porte sur la disponibilité du cluster lui-même.
- **Aucune métrique de ressources.** Ni CPU, ni mémoire, ni disque.
- **La disponibilité n'est pas mesurée.** Elle est déduite de la présence de logs, en staging seulement. Les logs de la production ne sont pas collectés ; seules ses traces le sont.
- **Aucune notification ne sort de Kibana,** et l'alerting ne se surveille pas lui-même.
- **Aucune rétention des logs.** Pas d'ILM ; les seuils disque d'Elasticsearch (85 / 90 / 95 %) mettront l'index en lecture seule bien avant que le volume de 5 Gio ne soit plein.
- **Sécurité d'Elasticsearch désactivée,** et Kibana accessible sans authentification derrière un simple `port-forward`.
- **Douze CVE de Spring Framework sont acceptées par exception** jusqu'au 31 décembre 2026 (§4.4). L'analyse est écrite ; elle n'est pas une preuve d'innocuité.
- **Aucun npm audit**, et la porte de qualité Sonar ne tourne que sur `main`.
- **Le comportement sous incident n'a été exercé qu'à la main.** Les échecs mesurés en CI portent sur l'accès au cluster, pas sur l'application : le rollback automatique de `deploy.sh` n'a été déclenché pour de vrai que lors de la campagne locale de `K8S.md` §14. Le rollback joué en production le 23 septembre appelait `rollback.sh` depuis `rollback-production`, un job manuel — ce n'est pas le même chemin de code.
- **La reconstruction d'environnement est jouée depuis un poste,** pas depuis un job de CI, et ne couvre pas la restauration de données, sans objet tant que la base vit en mémoire (`RELEASE.md` §9.5).
- **Les NetworkPolicy sont décrites, pas prouvées.** Le CNI par défaut de minikube ne les implémente pas et les ignore sans rien signaler. Celle d'APM Server n'est pas appliquée.
- **Le chart Helm n'a jamais déployé.** Il est rendu, linté, comparé au rendu Kustomize et accepté en `--dry-run=server`. Rien de plus.
- **La production n'existe pas.** L'environnement `production` vise le même minikube dans un autre namespace ; il démontre qu'un second environnement se décrit par les mêmes modules et d'autres valeurs, pas qu'une production existe.

9. Où retrouver les preuves

Ce rapport ne remplace pas les documents techniques du dépôt : il les résume. Chaque chiffre cité y est accompagné de la commande qui l'a produit et de la sortie observée.

Document	Ce qu'il porte
MONITORING.md	La stack ELK, les indicateurs DORA (§9), les traces et leur relevé (§10)
k8s/elk/alerting/README.md	Les huit règles, la justification de chaque seuil, les preuves de déclenchement
k8s/elk/alerting/rules/	Les règles elles-mêmes, une par fichier, requête comprise
k8s/elk/dashboards/README.md	Chaque panneau, confronté à l'agrégation Elasticsearch équivalente
back/config/dependency-check/suppressions.xml	Les douze exceptions Spring et leur analyse, condition par condition
.trivyignore.yaml	Les sept exceptions Trivy, justifiées et datées
RELEASE.md §7, §7.5 et §9.5	La procédure de release, la release 1.0.1, la reconstruction d'environnement
Projet GitLab public	Pipelines #2909284076 , #2912362926 , #2913490784 , Release v1.0.1 , artefacts des jobs
K8S.md §14	Les campagnes de déploiement réel, commandes et sorties
QUALITY.md	Les outils de qualité, leurs seuils, et ce qu'ils ont trouvé
SCRIPTS.md	Les scripts, leurs codes de sortie, et comment ils sont testés
TERRAFORM.md , ANSIBLE.md , HELM.md	L'infrastructure, ses frontières et ses limites assumées
ARCHITECTURE.md §9 à §11	Pourquoi l'option locale, sa transposition au cloud, et ses angles morts
documentation-infrastructure.md	Le document jumeau : architecture et procédures
docs/captures/	Les captures d'écran : pipelines, environnements, tableaux de bord, alertes

Sur docs/captures/ . Treize captures sont reprises dans ce rapport ; les quatre captures GitLab du 5 octobre ont été prises sans connexion, sur les pages publiques du projet. Les autres fichiers du dossier montrent des états antérieurs de la chaîne (pipelines et environnements de septembre, tableaux de bord du 19 septembre) et restent disponibles pour qui voudrait vérifier un pipeline étape par étape.

Reprises dans ce rapport	Section
kibana-dora-quatre-indicateurs-2026-10-05.png	§2.3
kibana-dora-non-mesurable-affiche-na-aout-2026-10-02.png	§2.3
gitlab-pipelines-liste-2026-10-05.png	§2.3
kibana-apm-services-2026-10-02.png	§5.3
kibana-apm-transactions-microcrm-2026-10-02.png	§5.3
kibana-securite-vulnerabilites-exceptions-erreurs-2026-10-02.png	§6.4
kibana-disponibilite-sondes-taux-demarrages-2026-10-02.png	§6.4
kibana-supervision-volume-latence-erreurs-2026-10-02.png	§6.4
kibana-alertes-liste-des-regles-2026-10-02.png	§6.5
kibana-alertes-suivi-declenchements-2026-10-02.png	§6.5
gitlab-release-v1.0.1-2026-10-05.png	§7.3
gitlab-pipeline-tag-v1.0.1-2913490784-2026-10-05.png	§7.3
gitlab-environnements-2026-10-05.png	§7.3

10. Glossaire

Terme	Signification
APM	<i>Application Performance Monitoring</i> : mesure du temps que met l'application à répondre, requête par requête.
Artefact	Fichier produit par un job du pipeline et conservé : un rapport, une image, un jar.
CI/CD	Intégration et déploiement continus : les contrôles et la livraison automatisés à chaque modification.
CVE	Identifiant public d'une faille de sécurité connue. Sa gravité va de LOW à CRITICAL.
CVSS	Note de gravité d'une CVE, de 0 à 10. Le pipeline s'arrête à partir de 7.

Terme	Signification
DORA	<i>DevOps Research and Assessment</i> : les quatre indicateurs de performance de livraison (§2.1).
ELK	Elasticsearch (stockage), Logstash ou Filebeat (collecte), Kibana (affichage) : la stack de supervision.
Image	Paquet contenant l'application et tout ce qu'il lui faut pour tourner.
Job, pipeline	Un job est un contrôle ou une action automatique ; un pipeline est la suite de jobs lancée par un commit.
KPI	Indicateur clé de performance.
MTTR	Temps moyen de rétablissement après un échec.
Namespace	Cloison logique d'un cluster Kubernetes : ici, un par environnement.
p50, p95, p99	Percentiles. « p95 = 5 ms » veut dire que 95 % des requêtes répondent en moins de 5 ms.
Pod	Unité d'exécution de Kubernetes : une instance en marche de l'application.
Registry	Entrepôt où sont rangées les images.
Release, tag	Un tag est une étiquette de version posée sur un commit ; la release est la version publiée qui en découle.
Rollback	Retour à la version précédente.
Runner	Machine qui exécute les jobs du pipeline.
Staging	Environnement de recette, où une version est essayée avant la production.
Trace	Enregistrement du parcours d'une requête dans l'application, avec la durée de chaque étape.